Gnuradio 系统平台总结

目录

一．综述：本文档将涵盖的内容 4

1. 本文档将会解决的问题 4
2. 本文档的逻辑结构 4
3. 本文档将来的工作 4

二．Gnuradio 平台代码逻辑结构 5

1. 从百模块园到代码书屋 5
   1.1. 界面到 python 代码 5
   1.2. 界面本身的界面代码 9
   1.3. python 代码到 C 代码 11
2. 如何从界面上的模块名字找到 C 代码 12
3. 如何阅读底层的 C 代码 13

三．Gnuradio 平台模块改写方法 14

1. C 语言入门 14
   1.1. C++基本数据类型和数据定义 14
   1.2. C++基本语法 15
      1.2.1. 如何定义变量
      1.2.2. 如何进行变量运算
      1.2.3. 如何写条件选择语句
      1.2.4. 如何写循环语句
   1.3. C++类 18
   1.4. C++类的高级操作 21
      1.4.1. 类的继承
      1.4.2. 类的虚函数
      1.4.3. 类的其他高级操作
2. PYTHON 入门 23
3. 如何创造全新代码模块 24
   3.1. 综述：如何创建全新代码模块 24
   3.2. 如何使用 python 语言创造全新模块 24
      3.2.1. 使用 python 创造全新模块原理
      3.2.2. 如何使用 python 创建新模块
      3.2.3. 以一个案例说明创建的流程
      3.2.4. 使用基于 python 新建模块的缺点
      3.2.5. 使用 PythonModule 的奇技淫巧
   3.3. 如何使用 C 语言在 gnu3.7.13.5 创造全新模块 28
      3.3.1. 创建 OOT 模型
      3.3.2. 向 OOT 模型中添加模块
      3.3.3. 更新与增添源代码
      3.3.4. 更新 xml 文件
      3.3.5. 在 gnuradio 中安装新模块
   3.4. 如何使用 C 语言在 gnu3.8.0 创造全新模块 36

4. 如何读懂原有模块的源代码并修改 36
4.1. 问题分析 37
4.2. 找到相关.h 和.cc 文件并分析其间联系 37
4.3. 自顶向下阅读代码 40
4.4. 修改 bug 重新编译 42

四．Gnuradio 平台 OFDM 相关模块代码实现原理 46

1. 发送端步骤 46
2. 发送端模块分析 47
3. 接收端步骤 58
4. 接收端模块分析 60

五．Gnuradio 平台上手学习指导（工作栈） 70

1. 如何安装 gnuradio 70
2. 如何跑通 demo 并通过小项目上手 gnuradio 70
3. 如何阅读源码成为一代宗师 71

六．如何将 SISO 系统改写为 MIMO 系统 71

1. 理论基础 72
1.1. 信道模型 72
1.1.1. 调制信道模型
1.1.2. MIMO 信道模型
1.2. 同步方法 75
1.2.1. SISO 系统的同步
1.2.2. MIMO 系统的同步
1.3. 信道估计方法 81
1.4. 信道均衡方法 82
2. 代码层软件仿真 82
3. 硬件层调试 83

Gnuradio 系统平台

一、 综述：本文档将涵盖的内容

本综述部分主要介绍本文档会解决的问题、以及本文档的逻辑架构，从而使得读者能够将本文档作为工具书，快速获得希望了解的问题。

1. 本文档将会解决的问题

本文档的基本定位是成为使用 gnuradio 进行通信平台的软件仿真的基本技术手册、同时也作为工具书和学习指导，从而辅助大家快速上手 gnuradio 之后，能够主动读懂 gnuradio 代码、通晓代码逻辑。（将会本着严谨可爱的态度撰写，因此珍惜“综述”这段严肃的文字吧！）

本文档将会解决以下四个问题：

问题一：gnuradio 平台的基本逻辑结构，其中包括：gnuradio 平台中，界面上的模块如何转换成 python 代码、界面上提供的模块对应于怎样的 python/C 代码、如何从界面的流程图看出底层代码在哪里和怎么实现；

问题二：gnuradio 平台中模块的改写方法，其中包括：每一个模块对应于怎样的 C 语言代码、C 语言入门、如何改写 C 代码或自己创造全新模块；

问题三：结合 OFDM 平台搭建的任务，介绍常用模块的 C 代码实现；

问题四：如何安装 gnuradio、如何通过简单小项目上手 gnuradio、在上手 gnuradio 后如何着手读源代码从而掌握 gnuradio。

2. 本文档的逻辑结构

本文档将会按照上述需要解决问题的顺序逐个回答。“二”中解决问题一——平台的基本结构；“三”中解决问题二——模块的改写方法；“四”中解决问题三——OFDM 系统中的常用模块原理；“五”中解决问题四——如何从 0 开始到精通。

3. 本文档将来的工作

实现多天线通信。

二、 Gnuradio 平台代码逻辑结构

1. 从“百模块园”（界面）到“代码书屋”（python→C）的编译流程？

进入 gnuradio 平台，直接输入 gnuradio-companion 就可以看到界面了，这是我们入手 gnuradio 后的第一印象——界面编程。但是实际上，界面编程仅仅是一个方便的手段，并不是 gnuradio 真正在底层运行的东西。大家都明白的道理，电脑上运行的东西，都必须是转换成代码的。

既然 gnuradio 平台上的这些漂亮的模块都必须转换成代码，那这个过程是怎样的呢？本部分就讲解这个问题。

1.1 界面到 python 代码（python 代码位置是.grc 文件的同文件夹下）

Python 这种语言非常直观、容易上手，这些优点决定了 python 特别适合界面编程，同时，linux 平台十分热衷于支持 python。Gnuradio 的原作者基于以上考虑，选择使用 python 作为界面到底层代码的第一次转换。下面讲这个第一次转换从哪里能够看出来、蕴含了什么信息。

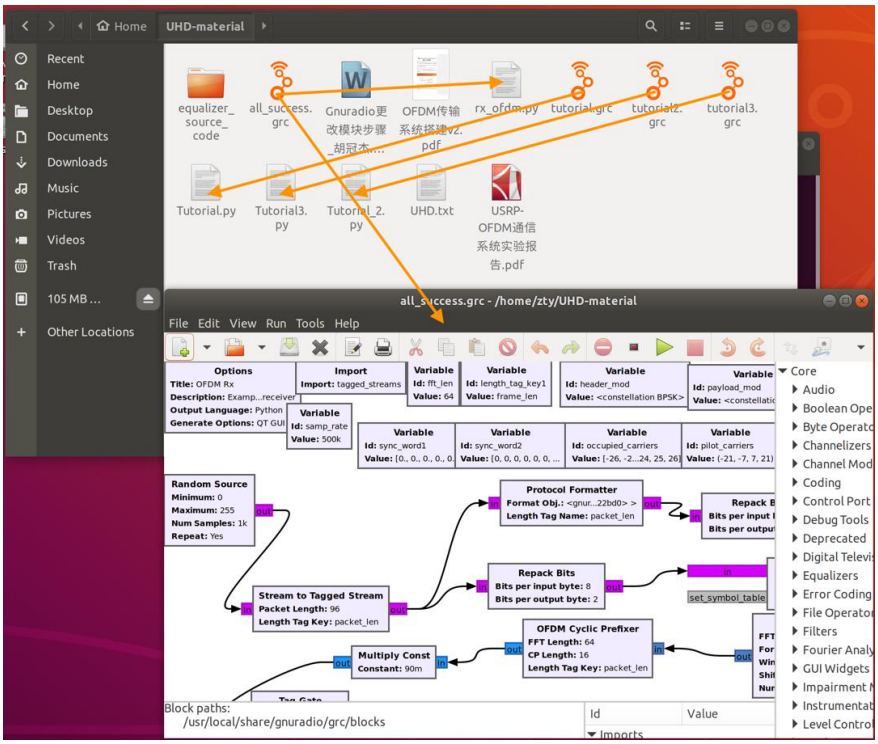


Fig 1. 本图展示了 grc 流程图文件产生的第一步转换出的 python 文件的对应关系。

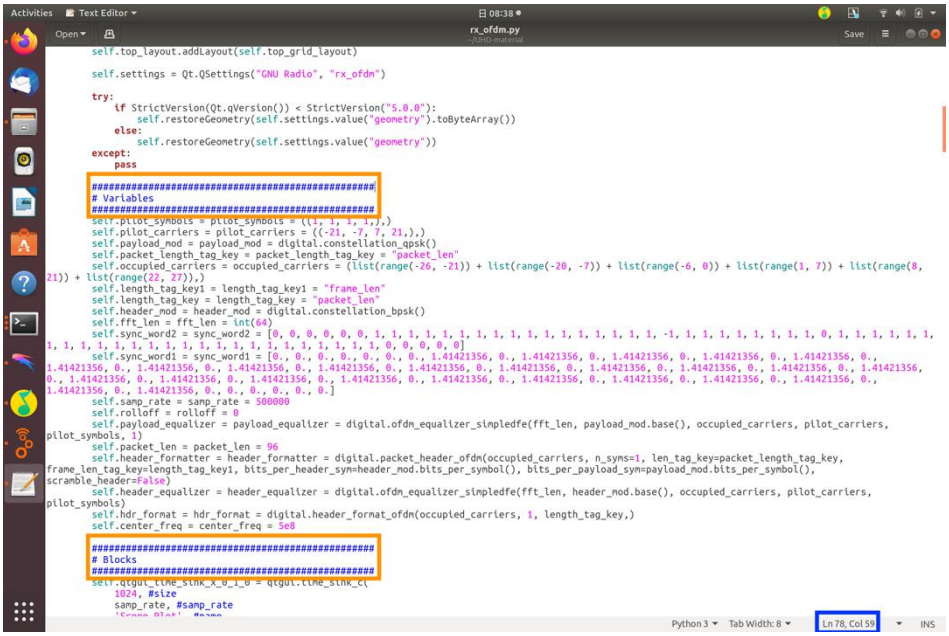
在上图 Fig 1.中，我用箭头标出了 grc 流程图文件生成 python 文件的对应关系，生成的过程是：我们首先拖动器件画出没有报错的流程图，之后单击流程图工具

栏里面代表运行的“黄色 ”（黄色右三角），就可以看到在流程图 grc 文件同一文件夹下生成了这个流程图对应的 python 文件，如果是自己建立的 grc 文件，一般而言 grc 生成的 python 文件是同名的，不同名字的时候怎么改正，参考 Fig 4.

我们现在继续进入由 all_success.grc 生成的 rx_ofdm.py 文件看一看这个 python 文件中主要记录了什么内容。双击 python 文件用文本文档 gedit 打开。

【首先声明：gnuradio 的作者是非常令我起敬的软件工程师，他的代码逻辑非常清晰，代码注释也都十分到位，代码规范堪称经典，代码布局和框架设计精巧，这是一位工程师能拿出的诚意，以便让使用者在理解代码的基础上进行通信系统设计。】

整体的 rx_ofdm.py 文件由三块蓝色注释分隔开，分别是 Variables 代码区域（第 78 行～第 99 行【gedit 右下角显示 Ln 就是现在光标所在位置的行数】）、Blocks 代码区域（第 101 行～第 374 行）和 Connections 代码区域（第 378 行～第 420 行）。



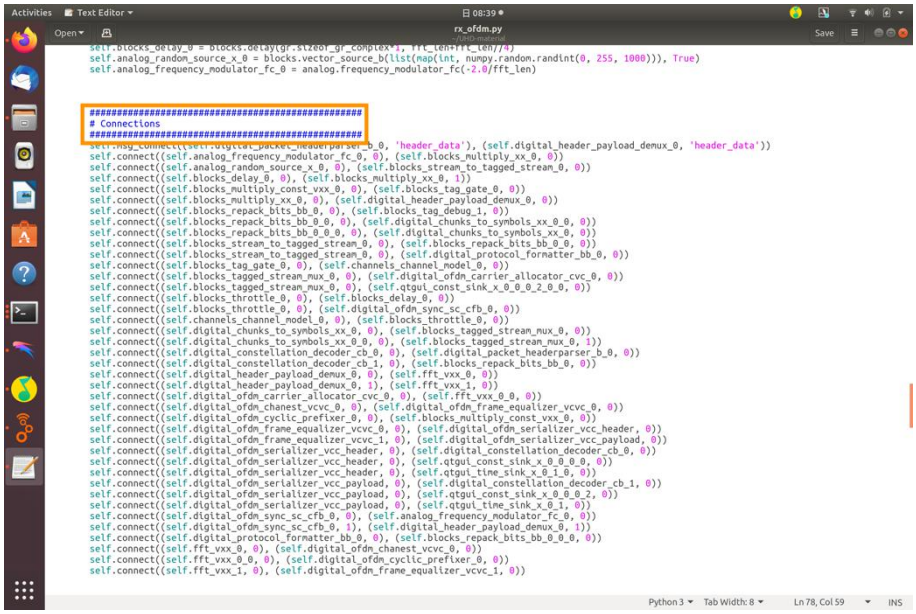


Fig 2. 上方的第一张图像中，位于图片中央的橙红色的两个框分别框出了上文说的 Variables 和 Blocks 代码区域的开头；第一张图片的右下角注意观察，我画出了一个深蓝色的小框，这个框里显示的 Ln78, Col59 的含义是这个文件中现在光标所在位置的行数（Line）以及列数（Column）；上方的第二张图像中橙红色框框出的就是 Connections 代码区域。

我们下面一个区域一个区域的介绍各个区域都是干什么的。

首先是 Variables 区域，这个区域是把这个流程图中所有的 Variables 模块中定义的变量转换成的 python 语句。这句话怎么理解呢，如下图所示，我用橙色箭头标出了前几个 Variables 在 grc 流程图中和 python 文件语句的对应关系：

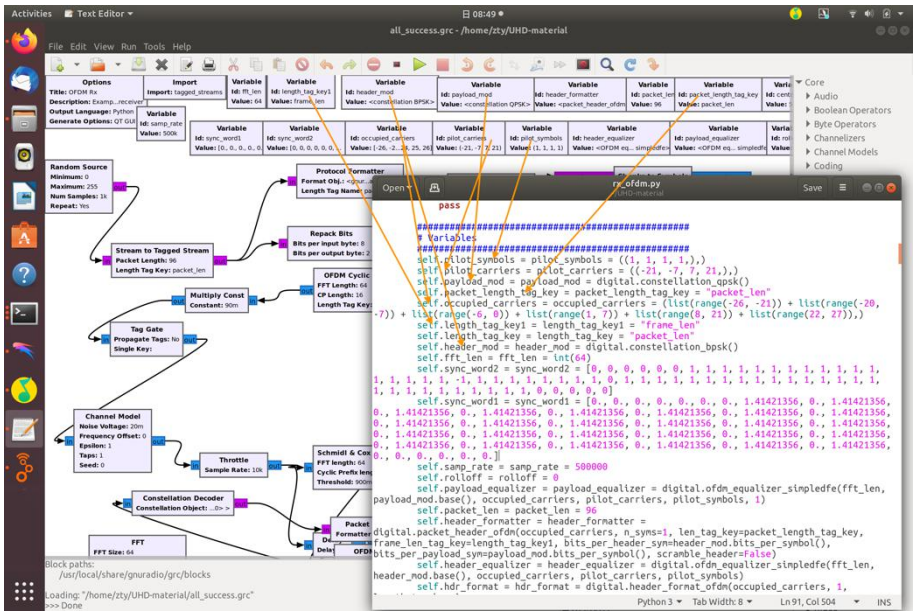


Fig 3. 上方的图像中我使用橙色箭头标出了流程图中的 Variables 模块中的变量和 python 文件中 Variables 区域中变量的对应关系，这样就体现了什么叫做流程图中的变量与 python 文件中的变量一一对应。用文字表述这句话就是——Variables 模块里面的 Id 字段的值和 python 文件里面的变量名称是一样的。

既然我们搞通了 Variables 部分，界面上的模块和 python 代码是一一对应的，那么以此类推，实际上 Blocks 这个部分，界面上的有功能的模块和 Blocks 里面的名称就是同样一一对应的，这一部分也就不再赘述了。

下面我们看一下 Connections 这个部分是个什么东西。

从 Fig 2.的第二张图里面我们可以看到 Connections 里面都是 self.connect(...)的格式，我就举用一个作为例子讲清楚 Connections 这个部分是干什么的：

```
>>self.connect((self.analog_random_source_x_0, 0), (self.blocks_stream_to_tagged_stream_0, 0))
```

上面这句话就是 Connections 部分的第三句，它的意思其实很好懂的，就是说把 analog_random_source 这个模块的输出，作为后一个 stream_to_tagged_stream 模块的输入，也就是说，self.connect(A, B)这个函数的作用就是顺序地连接 A、B 两个模块。

----------手动分割线----------

到这里为止，我们实际上已经搞明白了从 grc 文件单击 生成的 python 文件是什么意思了！很有趣，而且这也是我们破译 gnuradio 这个系统的突破口，下面的内容都是建立在对我上面说的内容充分理解的基础上的，还是请大家仔细阅读，搞通这一重逻辑关系。

----------手动分割线----------

但是，在读这个 python 文件的时候，可能大家会产生疑问：这个 python 文件只阐明了界面上的东西和 python 代码的对应关系，那这些模块的功能都在哪实现呢？

这个实际上就更深一层了，我们之后就会着手一层一层抽丝剥茧地找到最核心的 C 代码。（本部分结束）

【PS：这个 all_success.grc 生成的 python 文件名字为什么是 rx_ofdm.py 而不是 all_success.py，主要的原因是没有在代表流程图配置的模块——也就是 Options 模块，里面进行修改。修改的方法见下图 Fig 3.】

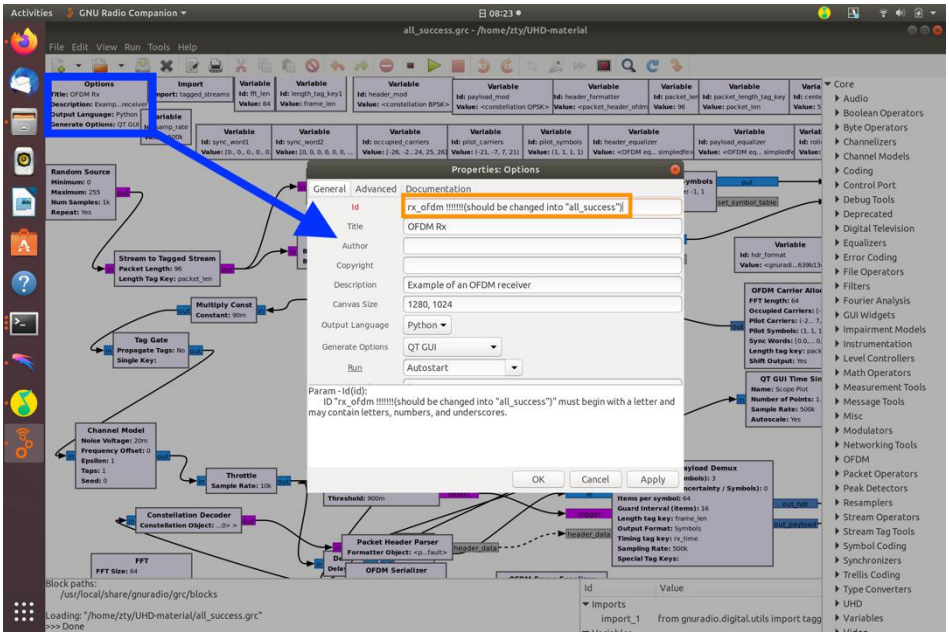


Fig 4. 如何修改从 grc 流程图文件生成的 python 文件的名字。

1.2 界面各个模块本身的界面代码（界面代码位置在/grc 文件夹下）

我们说：只要是能在电脑上运行的，不论是界面还是视频、图片、一切的软件，所有的只要能在电脑上跑的东西，必然是有它的代码的。基于这一点，grc 流程图界面必然是有代码的。

现在我们去找一找，这个模块界面代码在哪里（界面代码就是用来生成这种你能看见的长方形的代码）。

一般而言，这种界面的生成实际上使用的是诸如 css/qss/xml 之类的语言，gnuradio 生成代码的语言写在了.yml 文件里，用的是 yaml 语言。

下图 Fig 5.中展示的是 digital_ofdm_frame_equalizer_vcvc_block.yml 文件中的所有内容，这个 yml 文件对应的是 OFDM Frame Equalizer 模块，下面我讲解为什么这个文件就是 OFDM Frame Equalizer 模块，这个模块里面的东西都是什么。

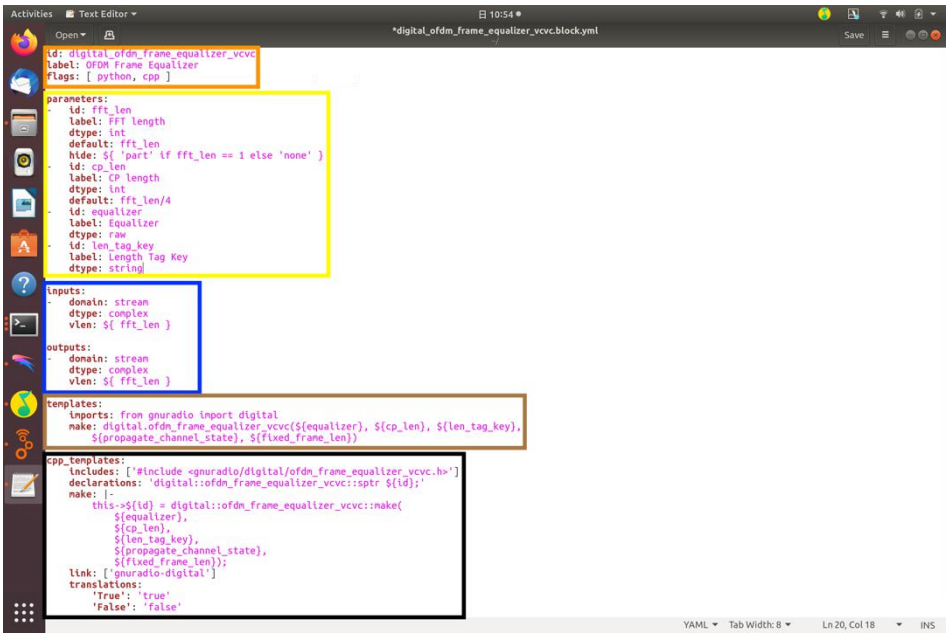


Fig 5. 上图是 digital_ofdm_frame_equalizer_vcvc_block.yml 文件的内容。

上图我用五个圈画成了五个部分（“通信界”的“奥运五环”，北京快要冬奥会了，祝愿天佑中华、顺利举办～）

橙色部分：id 是指这个模块在代码里面的唯一标识符，是给代码看的不是给人看的，别的代码想找它了，使用 id 来找的；label 是给人看的，就是显示在界面上的我们能看到的这个模块的名字（这里就是 OFDM Frame Equalizer 也就是我们看到的东西）；flags 的意思是表示为了执行这个模块的功能，底层代码使用的语言，好比这里是{python, cpp}，也就意味着底层使用了 python 和 cpp 代码，由于 python 代码是建立在 cpp 基础上的，因此最根本的还是 C 语言代码。

黄色部分：这一部分是 parameters 就是我们在这个模块的界面上输入的那些参数了，我们点开 OFDM Frame Equalizer 会看到如下图 Fig 6.所示的界面，这个界面上每一行就分别对应于 parameters 里面的东西。

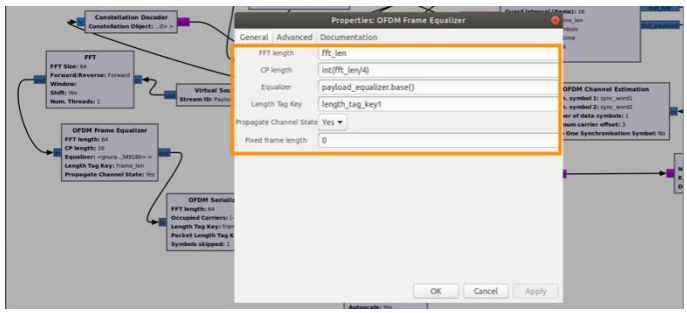


Fig 6. 上图是直接点开 OFDM Frame Equalizer界面模块的界面，那些写进去的参数就是.yml 文件里面的 parameters（就是黄色部分）。

蓝色部分：分别对应于这个模块的输入和输出，dtype 就代表输入和输出对应的类型，这里是 complex 那就是说类型就是复数；domain 是 gnuradio 自己定义的数据的格式，stream 是流，就是一个连续输入/出的时间序列；vlen 也就好解释了，就是这个流，一个块的长度，也就是 fft_len 了。总和上面三个，蓝色部分讲述的事情就是说：我的输入和输出都是长度为 fft_len 的复数连续序列。

绿色部分：templates 对应于执行这个模块功能的 python 代码文件是什么，这里显示的是 gnuradio.digital.ofdm_frame_equalizer_vcvc(…)，也就是说，如果我们现在想通过阅读 python 代码知道这个模块完成了什么工作、它的代码原理是什么，那么我们应该去找到 gnuradio 这个 python 包下面的 digital 文件夹，从这个文件夹中找出 ofdm_frame_equalizer_vcvc(…)这个“类”定义在哪里，阅读这个类的源代码，我们就应该知道这个模块究竟是干嘛的了。

黑色部分：cpp_templates 就是对应于执行这个模块功能的 C++代码文件是什么，这里显示的是#include <gnuradio/digital/ofdm_frame_equalizer_vcvc.h >，也就是说，如果我们想通过阅读 C（或 C++）代码知道这个模块完成了什么工作、它的代码原理是什么，那么我们应该去找到 gnuradio 这个 C 语言库下面的 digital 文件夹，从里面找到 ofdm_frame_equalizer_vcvc.h 头文件以及它对应的源文件。

1.3 python 代码到 C 语言代码

代码逻辑的最后一个部分了，坚持住，马上就通晓了～

现在需要追问一个问题，通过阅读 yml 文件，我们知道了某一个模块真正在执行功能的是 python 语言和 C 语言的底层代码。那么 python 和 C，哪个才是最根本的呢？毕竟一个体系中执行主要功能的代码一般而言是只有一种的，根本没必要在多种语言之间反复横跳，费这个事干嘛对吧。

这里我直接给出结论吧：在 gnuradio 平台下，最最本质、最最核心的代码是 C 代码，python 代码仅仅是作为 C 语言的扩展，是在安装 gnuradio 时系统根据 C 语

言代码自动生成的。这个过程解释起来就非常复杂了，涉及到很多编译器原理之类的理论，在此不再赘述。

至于怎么能知道这些,主要是阅读此类代码有一定的经验、在安装和编译 gnuradio 时仔细观察编译时那些“唰唰唰一行一行可快了”的编译代码，就得出这样的综合结论。

2. 如何从界面上的模块名字找到对应的底层 C 代码？

刚刚我们一直是在说咱们这个模块怎么从界面上对应到代码里面,这个过程相信大家也都知道的，咱们下面就去看看 C 代码怎么实现的。

先简单谈谈语言吧：

C 语言是当前世界上位于支配地位的语言，一方面具有底层语言的效率、同时具有高级语言丰富的库函数，可以说是代码语言之斗宗至圣。

python 语言则是与当前科研工作以及技术开发前沿结合最紧密的语言,虽然抛弃了运行效率，但是其可扩展性和系统适用性是代码语言中的无上至宝。

关于这两种语言的基本介绍，请参考“三.1”和“三.2”，那两个部分会从两种语言的基本数据结构开始、介绍函数、指针、数组、类、等等。

回到正题：

因为我们已经分析出来：C 语言写的代码是 gnuradio 最根本的核心，那么我们结合 C 语言的基本结构知道，我们应当找出我们希望分析的界面模块对应的.h 和.cc 文件（或.c/.cpp，无所谓的）。

找的方法就是依据“一.1.2 黑色部分”里面所说的，通过

#include <gnuradio/digital/ofdm_frame_equalizer_vcvc.h >

找出 gnuradio/digital/文件夹下的 ofdm_frame_equalizer_vcvc.h。在茫茫“码”海中寻找对的文件，如同大海捞针，所以我们首先应当熟悉一下 gnuradio 的文件结构。

下图是我做的一张示意图，展示 gnuradio 的文件架构：

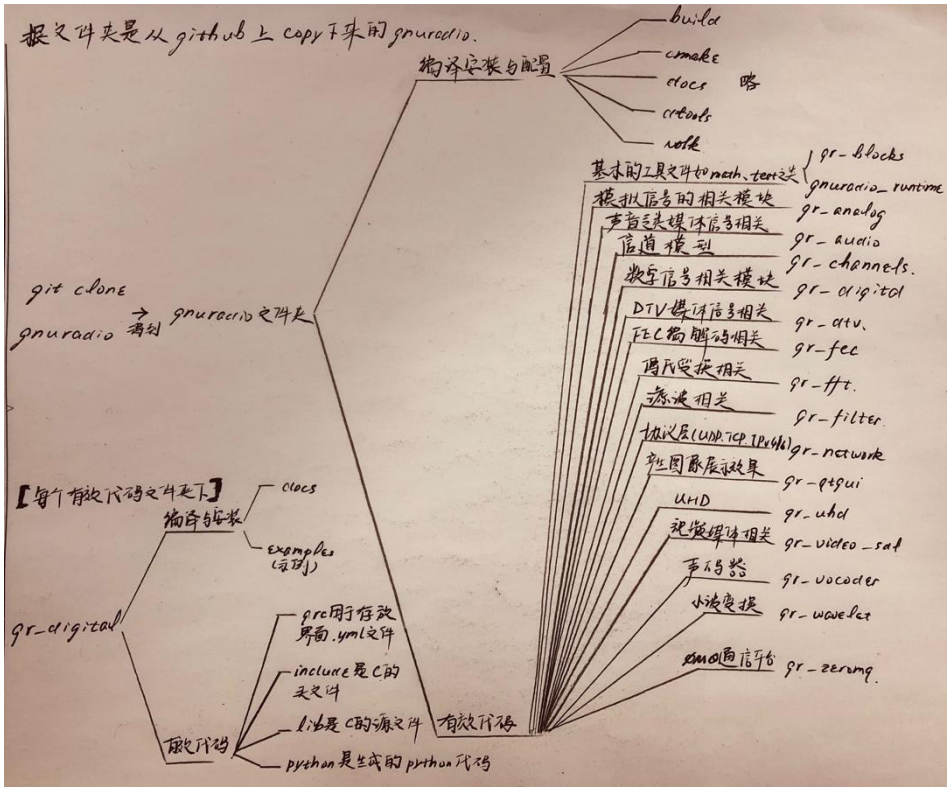


Fig 7. 上图是 gnuradio 的基本文件架构体系，最大的树形图描述的是 gnuradio根文件夹下第一层的子文件夹；左侧的小树形图描述的是每一个有效代码子文件夹下面的第一层孙文件夹的情况。

由上图的表示，假设我们需要寻找“ofdm_frame_equalizer_vcvc.h”，经过上文的提示，这个模块应当在“digital”文件夹下面，因此我们找到了 gr_digital 文件夹，单击进去。

因为我们寻找的是 C 语言代码，所以我们应该到 include 文件夹下去找头文件（.h），并且同时去 lib 文件夹下去找源文件（.c/.cc/.cpp）

经过一番寻找，我们能找到的名字之中含有 equalizer 的文件，头文件一共 5 个，源文件一共是 4 个，如果我们想要明白 OFDM Frame Equalizer 这个模块的基本原理，我们就应该进入头文件和源文件一共是 9 个文件，去读懂这 9 个文件了。

3. 如何阅读底层的 C 代码？

这一部分主要是提点大家如何细心地发现 C 语言代码的端倪。

阅读 C 语言代码，首先要读的是头文件。头文件的主要作用是：定义源文件中的“类”、定义源文件中需要用到的一些常量、以及解决源文件中的外部库函数调用。

阅读头文件的时候必须要关注几个要点：**首先是**头文件里面开头位置#include 了什么外部库文件，这个要求的目的是，万一读着代码突然看到了一个本头文件内从来没有出现过的函数/类的时候，迅速反应过来应该是调用了外部库文件。**第二是**头文件里面类和函数的定义。**第三是**头文件中常量的定义。

阅读 C 语言代码，第二要读的就是源代码。源代码读的时候也是有窍门的，源代码往往很多，很繁杂，其实更多的时候只需要寻找类定义中的个别函数就能够解决我们面对的问题，并不需要通读。

至于 C 语言入门和 python 语言入门，放在“三.1”和“三.2”里面讲解。

三、 Gnuradio 平台模块改写方法

1. C 语言入门（此处 C 语言默认包括 C++）

推荐网络教程：C++教程

推荐的学习路线：

首先花费一天左右的时间粗略阅读上述的教程，关注的重点是 C++中的基本数据类型怎么定义（尤其是指针和数组）、C++中的循环语句、条件选择语句、C++的“类”是什么、C++对“类”的高级操作。

第二开始在网上寻找一些由浅入深的小项目完成作为进阶的方法，从而深入理解第一步中阅读到的基础语法知识。这里推荐几个有趣的小项目：通过项目学习 C 语言（需要翻墙出去，没法翻墙的我可以帮大家翻）。

第三阅读 gnuradio 之类开源库的 C 代码，学习大佬们尤其是代码格式和代码风格如同 gnuradio 源代码一样 sexy 的代码去阅读，会学到很多东西。

下面是我自己写的对上述第一部分——基础知识的介绍，希望能够帮助大家快速入门。

1.1 C++基本数据类型和数据定义

C++是一个对于类型严格要求的语言，不同类型的数据原则上是不能直接进行数学运算的，就好比说我们有下面的代码片：

>>int a = 0;----------(compile success)

>>float b = 1.0; ----------(compile success)

>>int c = 0; ----------(compile success)

>>c = a + b; ----------(compile fail)

上面的例子中，第一句是定义一个整数 a 初始化为 0，第二句是定义一个浮点数 b 初始化为 1.0，第三句是定义一个整数 c 初始化为 0，第四句把 a 和 b 相加的结果赋值给 c，这个第四句话是无法通过编译的，因为 a+b 原则上得出的是一个浮点数，把一个浮点数赋值给整数，类型不对应。

下面这个表格是 C++（或 C 语言）中的基本类型

类型	位	范围
char	1 个字节	-128 到 127 或者 0 到 255
unsigned char	1 个字节	0 到 255
signed char	1 个字节	-128 到 127
int	4 个字节	-2147483648 到 2147483647
unsigned int	4 个字节	0 到 4294967295
signed int	4 个字节	-2147483648 到 2147483647
short int	2 个字节	-32768 到 32767
unsigned short int	2 个字节	0 到 65,535
signed short int	2 个字节	-32768 到 32767
long int	8 个字节	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807
signed long int	8 个字节	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807
unsigned long int	8 个字节	0 到 18,446,744,073,709,551,615
float	4 个字节	精度型占4个字节（32位）内存空间，+/- 3.4e +/- 38 (~7 个数字)
double	8 个字节	双精度型占8 个字节（64位）内存空间，+/- 1.7e +/- 308 (~15 个数字)
long double	16 个字节	长双精度型 16 个字节（128位）内存空间，可提供18-19位有效数字。
wchar_t	2 或 4 个字节	1 个宽字符

Fig 8. 上图是 C++基本类型，图源来自 C++教程

1.2 C++基础语法（条件、循环、函数）

C++语言的基本语法将会分几个部分介绍：首先介绍如何定义变量、之后是如何变量运算、再之后是如何写条件选择语句、最后是如何写循环语句。

1.2.1 如何定义变量

变量定义格式的语法公式如下：

类型名称 变量名称 = 初始化;

类型名称 变量名称;

举例如下：

定义一个字符（char）

```
char define_a_char = 'A';
```

或者不赋初值 char define_a_char;

定义一个双精度数（double）

```
double define_a_double = 10.157;
```

或者不赋初值 double define_a_double;

定义一个整数数组（int array）

```
int define_an_int_array[3] = {1, 2, 3};
```

或者不赋初值 int define_an_int_array[3];

定义一个指向整数的整形指针（int*）

```
int a = 10;

int* pointer_pointing_to_a = &a;
```

【指针定义语句解释：第一行先定义了一个整数 a，第二行定义了一个指向这个整数 a 的指针，初始化的地方写的是&a，这个&的意思是“取地址”，也就是事实上 pointer_pointing_to_a 本质上是内存里面的一个地址，这个地址下存储的东西是一个整数，这个整数的名字是 a】

1.2.2 如何进行变量运算

进行变量运算非常简单，和数学一样的，注意类型一致就行。一般而言，基本类型（Fig 8.里面给出的东西），都具有加减乘除整除取余这 6 种运算的，但是如果是你自己定义的类型想具有加减乘除这些基本运算的话，需要**重载运算符**，这个用法比较高级，不算在本入门教程的讨论范围内。

1.2.3 如何写条件选择语句

选择语句有两种：第一种是基本的 if{…}else if{…}else{…}；第二种是 switch (某个变量 A)：case A 的第一种取值：…; case A 的第二种取值：…;……default：…

下面分别展示其格式：

第一种 if-else 结构：

```
int a = 10;
int b = 20;
if (a > 5){
b = b + 1;
}else if (a < 2){
b = b + 5;
}else{
b = 0;
}
```

第二种 switch-case 结构：

```
//前面定义了一个整数 a，a 可以是 0，1，2
switch (a){
    case 0:
        printf("a 取值为 0\n");
        break;
    case 1:
        printf("a 取值为 1\n");
        break;
    case 2:
        printf("a 取值为 2\n");
        break;
    default:
        printf("a 的取值不在 0，1，2 范围内，请检查你的程序\n");
```

```
        break;
}
```

1.2.4 如何写循环语句

循环语句一共有两种，分别是 for 循环、while 循环。

第一种 for 循环：

```
for (int i = 0; i < 10; i++){
    print("你好\n");
}
```

第二种 while 循环：

```
int i = 0;
while (i < 10){
    printf("你好\n");
}
```

1.3 C++类

一般而言，C++的类应该是在头文件中声明，在源文件中定义。

这一部分博大精深，耐人寻味，以本人的实力感觉无法讲解的出神入化，这一部分更多还是需要靠自己感悟、靠不断的练习去体会这种快乐。因此这一部分我只讲解语法公式的通俗理解，并不举例。具体的例子建议去阅读 C++教程之后去做练习。

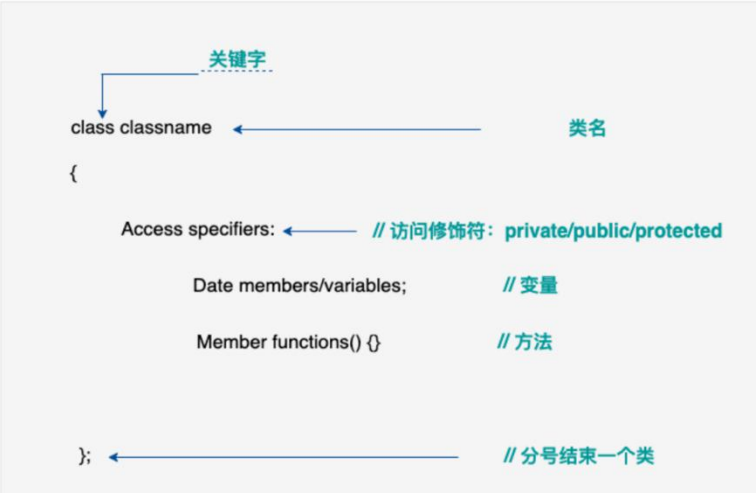


Fig 9. 上图是 C++类的定义语法公式，图源来自 C++教程

这个语法公式的通俗理解如下：

我自己定义了一个类，所以我先写一个 class，这个类你得给他取个名字，好比我定义一个“人”类，那就应该用如下开头方式：

```
class human{
    ........(咱们还没写的东西);
}
```

下面咱们想一下，“人”得有啥，必须得有器官、得有肉、得有骨头，同时“人”这个概念是抽象的，并不是特指的，不仅我是人、你也是人、他也是人，并不特指，但是对于器官、肉、骨头这些是针对每一个人的个体，他分别自己具有的，别人拿也拿不走。这就叫做私有变量。

```
class human{
    private:
        bone my_bone;
        flesh my_flesh;
        organ my_organ;
    .......(咱们还没写的东西)
}
```

【至于这里为啥我把 bone/flesh/organ 也当成类型，因为单说骨头、肉、器官实际上也是抽象概念，也不是特指某个人的骨头、肉、器官。但是 my_bone/my_flesh/my_organ 是类的实例化，就变成特指就是我的骨头/肉/器官】

我们再接着想，“人”这个类别有些功能，大家都能用。好比说我毕业了去打工，那我作为一个人，我能工作，工作就是我的一个函数，这个函数可以被别人调用，这就是打工。说难听点，那就是说：我有一个函数叫做工作，别的人都可以调用我这个函数，从而获得效果，这里我就被资本家剥削了，资本家获得了我的劳动成果，但是为了获得这个劳动成果，资本家要给我一点生产原料，这就很形象了——资本家就是别的人，给我的生产原料就是我拥有的“工作”这个函数的输入，资本家拿走的就是我的“工作”函数的返回值。因为我的“工作”函数可以**被别人调用**，所以这个函数是一个公有函数。

下面举一个私有函数的例子，我再怎么社畜，我也得有自己的生活，饭得自己吃、饮料得自己喝，这些函数对于外界是不能使用的，只能是我自己用！好比我在定义我自己的“工作函数”时，里面可以用“吃饭函数”“喝饮料函数”，但是资本家不能调用我的“吃饭函数”“喝饮料函数”！

所以我们把公有的工作函数和私有的吃饭、喝饮料函数加到我们的类里面去，那就是下面的语法公式了：

```
//“人”类是“动物”类的继承者，
//继承了 animal 类中的 eat()和 drink()函数
class human : public animal
{

    //Declare private variables and functions
    private:
        bone my_bone;
        flesh my_flesh;
        organ my_organ;
        void eat_food_myself(food my_lunch);
        void drink_beverage_myself(beverage my_juice);

    //Declare public variables and functions
    public:
        value work(source input);

};//The declaration of class human ends

//The definition of class human starts
//Construction function（构造函数）
human::human(bone_init, flesh_init, organ_init) :
    /my_bone(bone_init),
    /my_flesh(flesh_int),
    /my_organ(organ_init){
}

//Deconstruction function（析构函数）
human::~human(){
}

//Private function definition
//Definition of "eat_food_myself"
void human::eat_food_myself(food my_lunch){
    //“人”类继承的“动物”类的 eat()函数
    eat(my_lunch);
    return;
}

//Definition of "drink_beverage_myself"
void human::drink_beverage_myself(beverage my_juice){
```

```
    //“人”类继承的“动物”类的 drink()函数
    drink(my_beverage);
    return;
}


//Public function definition
//Definition of "work"
value work(source input){
    return input;
}
```

1.4 C++类的高级操作（类的继承、类内的虚函数）

1.4.1 类的继承

语法公式如下

```
class <派生类名>:<继承方式 1><基类名 1>,<继承方式 2><基类名 2>,…
{
<派生类类体>
};
```

继承这里概念很复杂，严谨的定义请参考网络教程，这里我讲一下不完全正确的通俗理解：就是类的继承就是父亲和儿子的关系，被继承的类就是爹、继承得到的类就是儿子，儿子具有他爹的所有函数以及变量（实际上不正确，至于儿子究竟具有他爹的什么，要看继承方式，如果是爹爹的东西全都能被儿子拿走，这叫 public 继承，如果爹爹的自己的东西不能被儿子拿走（就是基类中使用 private 修饰符修饰的东西），那这就叫做 private 继承，还有一种继承方式叫做 protected 继承，这个用法相对高级，不是本文档的任务）。

1.4.2 类内的虚函数

虚函数也是只讲解通俗理解，就是老子没完成的事情儿子完成，老子的儿子没完成的事情孙子完成，愚公移山，“子子孙孙无穷匮矣”，直到完成为止。（注意，虚函数分为“虚函数”和“纯虚函数”，虚函数是老子好歹干了一点点（就是这个函数，老子还是姑且定义过的），如果老子完全没干（就是函数完全没有定义、只有声明），那就是纯虚函数）

假设 Granpa 这个类别里面有两个虚函数分别叫做“enter_tsinghua()”和“enter_FeiLab()”。我爷爷没考上清华，更没进高老师实验室，所以两个函数都没实现；假设说我爸考上了清华，但是没有进入高老师实验室，所以我爸实现了 enter_tsinghua()函数，但是没实现 enter_FeiLab()函数；最后是我，我不仅上了清华，而且还进了 FeiLab，所以我在我爸实现了 enter_tsinghua()函数的基础上，又实现了 enter_FeiLab()函数。

那么上述过程的语法公式应该是如下所示的：

```
class Grandpa : public human
{
    public:
        //enter_tsinghua 是一个虚函数，因为爷爷有定义
        virtual void enter_tsinghua(){return Join_gaokao();};
        //enter_FeiLab 是一个纯虚函数，因为爷爷没有定义
        virtual void enter_FeiLab() = 0;
};

//爸爸类继承爷爷类
class Father : public Grandpa
{
    public:
        void enter_tsinghua();
};

//爸爸考上了清华，所以这个 enter_tsinghua()函数在爸爸这里被重新定义了，
//但是爸爸仍然没定义 enter_FeiLab()
//Definition and instantiation of "enter_tsinghua()"
void Father::enter_tsinghua(){
    int gaokao_score = 0;
    gaokao_score = Join_gaokao();
    if (gaokao_score > 700){
        print("成功考上清华！");
    }
    else{
        printf("高考失败，去复读上了清华！");
    }
}

//儿子类继承爸爸类（自然也就二次继承了爷爷类）
class Son : public Father
{
```

```
    public:
        //儿子类实现了纯虚函数 enter_FeiLab 的定义
        void enter_FeiLab();
};

//Definition and instantiation of "enter_FeiLab()"
void Son::enter_FeiLab(){
    print("进入高老师实验室学习通信！");
}
```

1.4.3 （抽象类）（模版类）（类多态）（数据封装）请对照网络教程自学

2. python 入门

python 语法相比于 C++已经是很大程度的精简，因此我就不在这里介绍了，仅仅推荐学习路线和学习方法。

推荐网络教程： python 教程

推荐的学习路线：

首先花费一天左右的时间粗略阅读上述的教程，关注的重点是 python 中如何调用包、如何写 python 类；

第二阅读 gnuradio 之类开源库的 python 代码，学习大佬们尤其是代码格式和代码风格如同 gnuradio 源代码一样 sexy 的代码去阅读，会学到很多东西。

最后十分建议学 python 的同时在 ubuntu 系统上学 python，因为 python 是一种“胶水语言”，里面有无数的实用包、怎么配置这些包的环境、怎么以一种工程的视角去看软件工程，都可以在这个过程中获得提高（~~虽然最后发现所谓软件工程就是七分配环境、二分靠算法、一分靠灵感~~【别骂了别骂了，我要脸呜呜呜.jpg，手动滑稽】）。

3. 如何创造全新模块代码？

3.1. 综述：如何创建全新模块

创建全新模块一共是有两种方法，第一种是直接使用 python 语言进行修改，使用 python 语言重新创建全新模块；第二种是使用重新从 C++源代码开始，写 C

源文件和头文件，之后写 xml 或 yml 配置文件，再之后注册配置文件，最后重新加载 make build 获得更新后的代码库。

这里我们两种主流的方法都会介绍，下面从如何使用 python 语言创建全新模块开始介绍；之后介绍如何在 ubuntu 16.04+gnuradio 3.7.13.5 平台配置下实现修改 OOT 模型添加新模型；再之后介绍如何在 ubuntu18.04+gunradio 3.8.0 平台配置下实现修改 OOT 模型添加新模型。

3.2. 如何使用 python 语言创建全新模块

3.2.1.使用 python 创建新模块的原理

为什么能够使用 python 创建新模块呢？因为实际上在 gnuradio 进行 make 之后，源代码的 C 就已经被转换成了 python 的包，是允许使用类似如下的代码进行调用的：

```
>>import gnuradio as gr
>>import gnuradio.digital as digital
>>constel_map = digital.constellation
```

上面的代码显示了调用 gnuradio 的 digital 模块中的 constellation 星座图模块展示星座图。我举一个完整的使用 python 文件编写 gnunradio 的方法（也就是说，根本不需要拉动模块的流程图，直接手动编写 python 脚本就能实现一个通信系统！）

```
#! /usr/bin/env python

from gnuradio import gr
from gnuradio import audio, analog

class dial_tone_python(gr.top_block):

    def __init__(self):
        gr.top_block.__init__(self)

        sample_rate = 32000
        ampl = 0.1

        src0 = analog.sig_source_f(sample_rate, analog.GR_SIN_WAVE, 350, amp
l)
```

```
        src1 = analog.sig_source_f(sample_rate, analog.GR_SIN_WAVE, 440, amp
l)
        dst = audio.sink(sample_rate, "")
        self.connect(src0, (dst, 0))
        self.connect(src1, (dst, 1))

if __name__ == "__main__":
    try:
        dial_tone_python().run()
    except KeyboardInterrupt:
        pass
```

上面这个模块运行之后就会发出电话拨号的声音，这就是说明了其实要想使用 gnuradio 文件的话，可以抛弃流程图，直接使用 python 代码调用 gnuradio 库里面的模块。这也就证明了 gnuradio 的流程图也就是一个壳子，我们可以通过修改里面真正执行功能的代码从而生产新的模块。下面 3.2.2 主要讲解基本流程，3.2.3 使用一个例子说明，图文结合。

3.2.2.如何使用 python 创建新模块

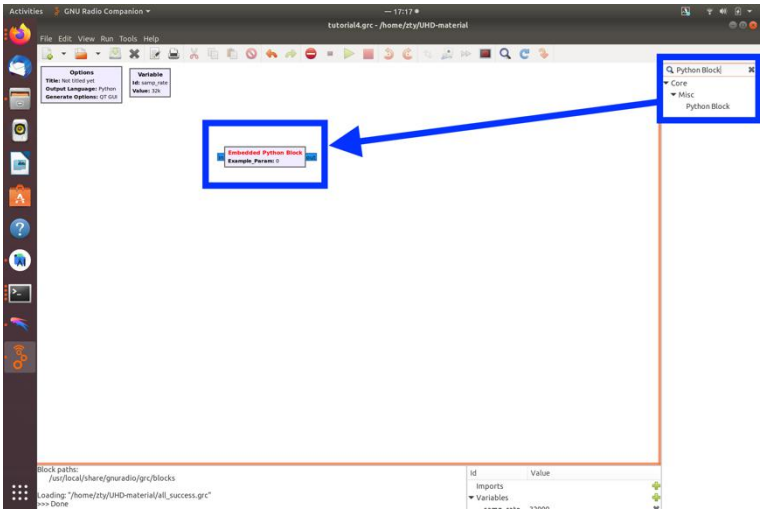


Fig 9. 上图是 Python Block 模块的位置

首先是在模块库里面搜索 Python Block 从而找到使用 python 代码构建流程图的 Embedded Python Block 模块，双击打开其参数界面，打开后如下图所示：

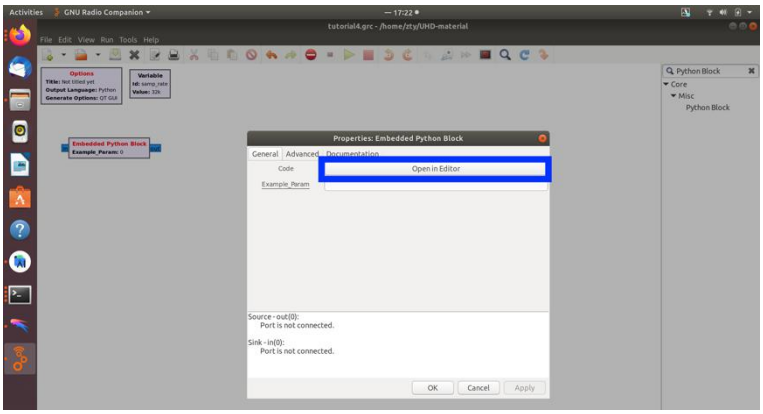


Fig 10. 上图是 Python Block 模块打开后的情形，蓝色框框出的地方有一个按钮，按钮上面写的是 Open in Editor（使用编辑器打开），单击之。

第二是单击上述图 Fig 10.中的 Open in Editor 按钮，如果是初次使用，单击这个按钮的时候会出来一个信息弹窗问你想要使用什么编辑器，直接使用 Default 就行，当然也可以自己选择。

第三是在编辑器中修改代码，实现我们自己需要的功能。下图就是修改的方法，和各行代码的解释：



Fig 11. 上图是 Python Block 模块打开并修改后的情形，里面详细解释了各行都是什么含义，并且给出了案例。

3.2.3. 以一个案例说明创建的流程

这里给大家提供一个教程，如何使用 python 语言建立自己的模块，如下链接自己建立基于 python 语言的模块案例。整个过程非常轻松愉快，大家上手也很容易啦！

这里我在 ubuntu16.04+gnuradio3.7.13.5 以及 buntu18.04+gnuradio3.8.0 上都测试过了，都是没有任何问题的，请大家放心食用。

3.2.4.使用基于 python 新建模块的缺点

使用这个方法新建模块的最大缺点就是这个创建出来的模块不会被注册到模块库里面去，要是想移植这个咱们刚刚自己定义的模块的话，只能是把咱们刚刚自己写的 python 文件拷贝过去。

除了这个缺点以外真的没啥不好的，又方便还避免了改写 C 语言一大串的痛苦。

3.2.5.使用 Python Module 的奇技淫巧

我们在模块的查找栏里面输入 Python 的话，不仅会出来一个 Python Block，还会出来一个 Python Module，下面给大家讲一下这个 Python Module 是啥意思。

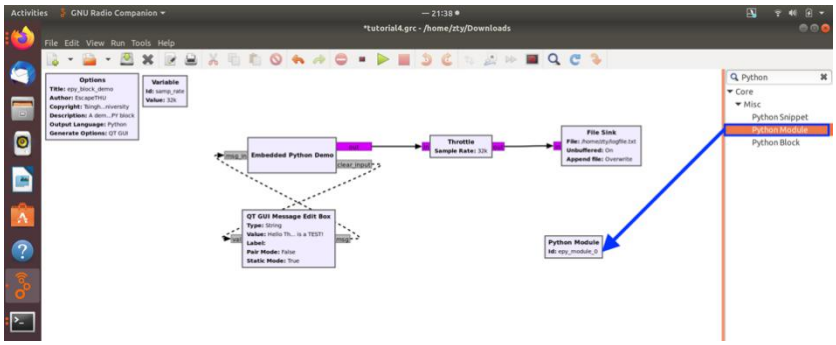


Fig 12. 上图是 Python Module 的情况。

这个模块的意思是，在里面定义一些函数、变量，这些函数可以在流程图文件的其他地方被自由地调用。就好比说，如果我在 python module 的.py 文件里面写了下面的内容

```
#假设在 Python Module 模块的 staff.py 文件里面
#写了下面的内容(这个.py 文件的名字 staff.py 是自己定的，叫啥都行)
a = 2

def double(arg):
    return 2 * arg
```

那么在另一个模块里假设需要用到 a 这个参数的值，那么对应的位置就可以填上 staff.a 就可以了，假设另一个模块需要使用 staff.py 里面的 double 函数，那就可以直接写 staff.double(...)就可以调用。

以上就是全部的使用 python 自定义模块的内容，下面介绍使用 C++语言分别在 gnu3.7.13.5 和 gnu3.8.0 里面实现新模块（二者不同！）

3.3. 如何使用 C 语言在 gnu3.7.13.5 创建新模块

3.3.1. 创建 OOT 模型

打开 ubuntu 终端，输入指令 gr_modtool nm tutorial 来创建 gr-tutorial 文件夹；而后输入指令 ls 可以看到在 Home 文件夹中已经建立了这个文件夹，具体操作见下图：

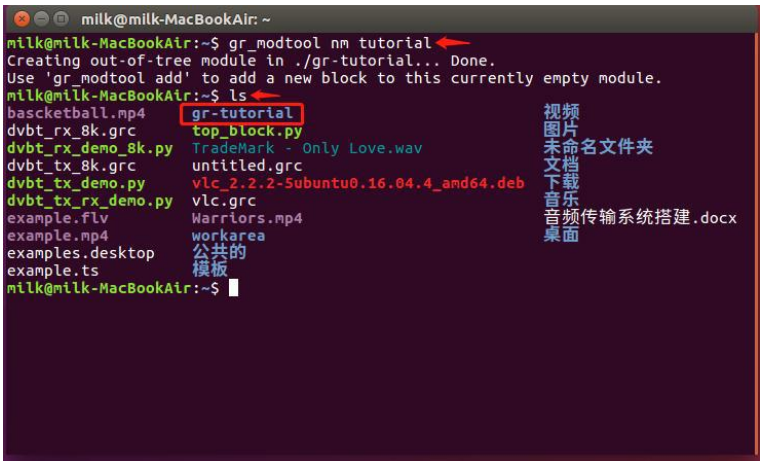


Fig 13. 上图是新建新模块文件夹。

通过指令 cd gr-tutorial 进入文件夹内部，输入 ls 指令查看文件夹中内部构成

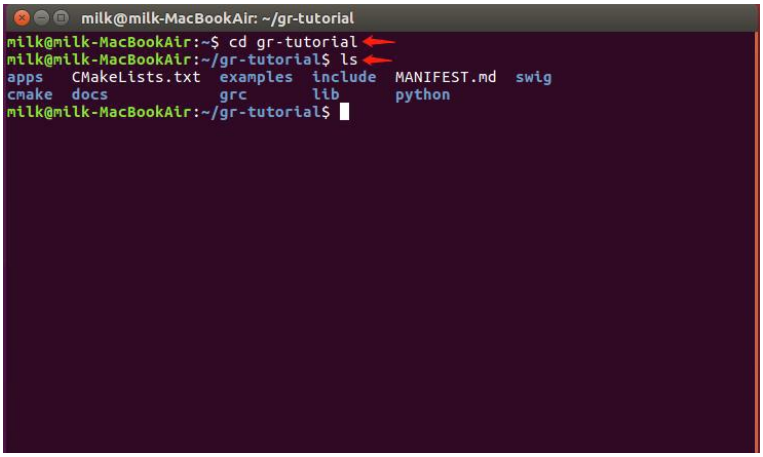


Fig 14. 观察文件结构。

3.3.2. 在 OOT 模块中插入 My QPSK Demodulator 模块

首先输入 gr_modtool add my_qpsk_demod_cb 指令，my_qpsk_demod_cb 代表这个模块的类名称，‘cb’ 代表这个模块为复杂数据(complex data)输入，spite byte(字节)输出；然后我们需要选择我们的模块类型，这里我们选择的是 general 类型，也就是常规类型；语言我们选择 cpp；最后，这个模块需要一个用户界面，对于 my_qpsk_demod_cb 来说，gray_code 被设置为一个默认参数。上述流程如下图所示：

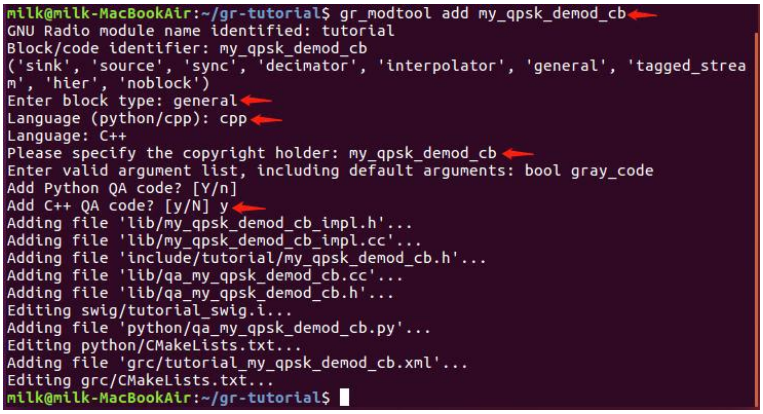


Fig 15. 编写 OOT 文件。

3.3.3. 更新以及增添代码

更新以及增添代码性需要我们打开文件 lib

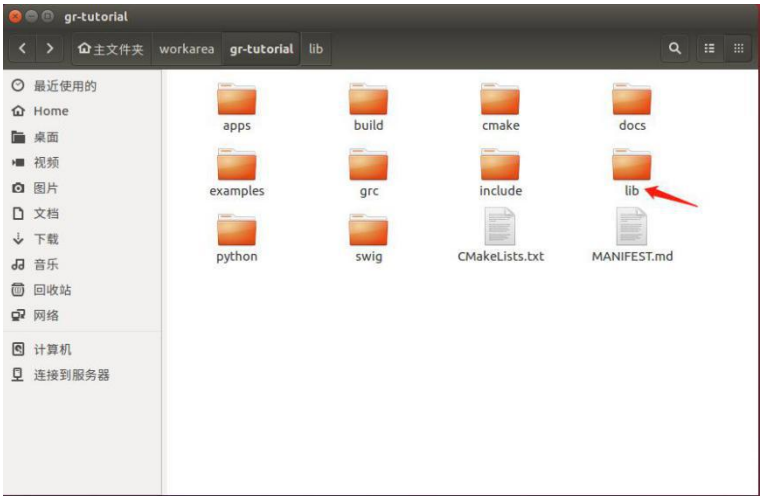


Fig 16. 更改.h 和.cc 文件从而实现模块的功能。

在 .cc 和 .h 这两个文件中进行代码更改

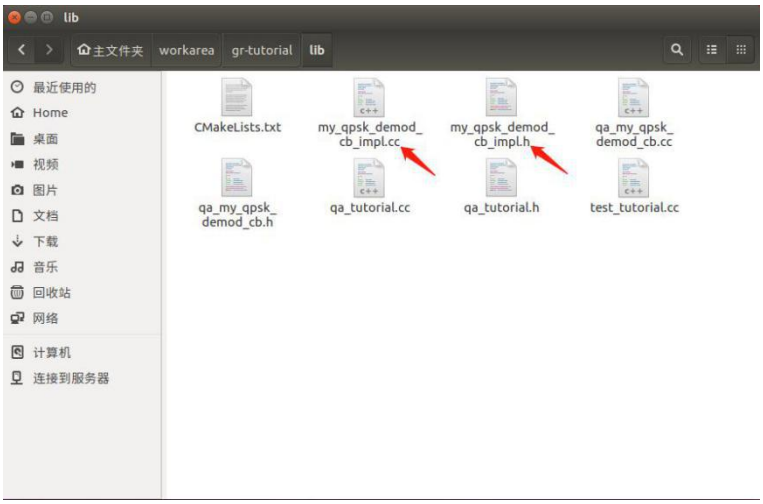


Fig 17. 更改.h和.cc文件从而实现模块的功能。

更新代码部分（my_qpsk_demod_cb_impl.cc）：

```
/*
 * The private constructor
 */
my_qpsk_demod_cb_impl::my_qpsk_demod_cb_impl(bool gray_code)
  : gr::block("my_qpsk_demod_cb",
              gr::io_signature::make(<+MIN_IN+>, <+MAX_IN+>, sizeof(<+ITYPE+>)),
              gr::io_signature::make(<+MIN_OUT+>, <+MAX_OUT+>, sizeof(<+OTYPE+>)))
{}

/*
```

更新前

```
/*
 * The private constructor
 */
my_qpsk_demod_cb_impl::my_qpsk_demod_cb_impl(bool gray_code)
  : gr::block("my_qpsk_demod_cb",
              gr::io_signature::make(1, 1, sizeof(gr_complex)),
              gr::io_signature::make(1, 1, sizeof(char))),
    d_gray_code(gray_code)
{}

/*
```

更新后

其中，<MIN_OUT> 和 <MAX_OUT>表示端口的最大值和数量。<ITYPE> 和 <OTYPE>表示输入和输出端口的数据类型，需要手工填写。

然后我们需要在头文件（my_qpsk_demod_cb_impl.h）中将其声明为类的私有成员

```
private:
 // Nothing to declare in this block.
```

更新前

```
private:
 bool d_gray_code;
```

更新后

而后转入（my_qpsk_demod_cb_impl.cc）文件中，更改函数 general_work，如下所示：

```
int
my_qpsk_demod_cb_impl::general_work (int noutput_items,
                   gr_vector_int &ninput_items,
                   gr_vector_const_void_star &input_items,
                   gr_vector_void_star &output_items)
{
  const <+ITYPE+> *in = (const <+ITYPE+> *) input_items[0];
  <+OTYPE+> *out = (<+OTYPE+> *) output_items[0];

  // Do <+signal processing+>
  // Tell runtime system how many input items we consumed on
  // each input stream.
  consume_each (noutput_items);

  // Tell runtime system how many output items we produced.
  return noutput_items;
}
```

更新前

```
int
my_qpsk_demod_cb_impl::general_work (int noutput_items,
                   gr_vector_int &ninput_items,
                   gr_vector_const_void_star &input_items,
                   gr_vector_void_star &output_items)
{
  const gr_complex *in = (const gr_complex *) input_items[0];
  unsigned char *out = (unsigned char *) output_items[0];
  gr_complex origin = gr_complex(0,0);
  // Perform ML decoding over the input iq data to generate alphabets
  for(int i = 0; i < noutput_items; i++)
  {
      // ML decoder, determine the minimum distance from all constellation points
      out[i] = get_minimum_distances(in[i]);
  }
  // Tell runtime system how many input items we consumed on
  // each input stream.
  consume_each (noutput_items);
  // Tell runtime system how many output items we produced.
   return noutput_items;
}
```

更新后

这个 general_work 函数调用了 get_minimum_distances(const gr_complex &sample)，该函数需要我们添加（单独添加）：

```
unsigned char
my_qpsk_demod_cb_impl::get_minimum_distances(const gr_complex &sample)
{
  if (d_gray_code) {
     unsigned char bit0 = 0;
       unsigned char bit1 = 0;
       // The two left quadrants (quadrature component < 0) have this bit set to 1
       if (sample.real() < 0) {
          bit0 = 0x01;
       }
       // The two lower quadrants (in-phase component < 0) have this bit set to 1
       if (sample.imag() < 0) {
         bit1 = 0x01 << 1;
       }
       return bit0 | bit1;
  } else {
    // For non-gray code, we can't simply decide on signs, so we check every single quadrant.
    if (sample.imag() >= 0 and sample.real() >= 0) {
         return 0x00;
    }
    else if (sample.imag() >= 0 and sample.real() < 0) {
                return 0x01;
    }
    else if (sample.imag() < 0 and sample.real() < 0) {
         return 0x02;
    }
    else if (sample.imag() < 0 and sample.real() >= 0) {
         return 0x03;
    }
  }
}
```

函数声明同样要添加到类标题中（my_qpsk_demod_cb_impl.h），如下：

```
    public:
     my_qpsk_demod_cb_impl(bool gray_code);
     ~my_qpsk_demod_cb_impl();
```

更新前

```
public:
 my_qpsk_demod_cb_impl(bool gray_code);
 ~my_qpsk_demod_cb_impl();
 unsigned char get_minimum_distances(const gr_complex &sample);
```

更新后

最后需要更改 forecast()函数模块，系统需要知道需要多少数据才能确保每个输入数组的有效性。如前所述，forecast()函数提供了这些信息，因此必须进行修改。

```
void
my_qpsk_demod_cb_impl::forecast (int noutput_items, gr_vector_int &ninput_items_required)
{
  /* <+forecast+> e.g. ninput_items_required[0] = noutput_items */
}
```

更新前

```
void
my_qpsk_demod_cb_impl::forecast (int noutput_items, gr_vector_int &ninput_items_required)
{
unsigned ninputs = ninput_items_required.size ();
for(unsigned i = 0; i < ninputs; i++)
  ninput_items_required[i] = noutput_items;
}
```

更新后

3.3.4. 更新 XML 文件

这个 XML 文件使得使用者能够在 GRC 中调用这个 OOT 的模块以及它的源代码。此外 XML 文件定义了接口来传递模型的具体参数。为了能够在 GRC 中使用一个模型，我们还需要手动修改这个 XML 文件。我们之前创建的模块被命名为 tutorial_my_qpsk_demod_cb.xml，同时被放置在 grc 文件夹下。

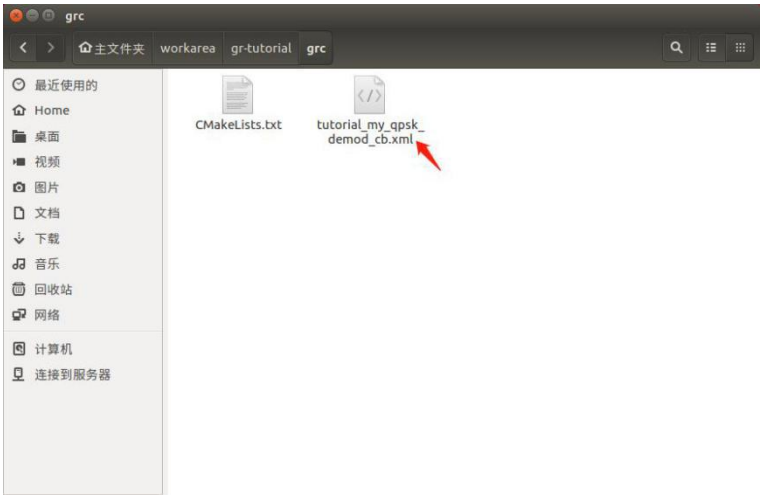


Fig 18. 更改.xml文件从而把新的模块注册到配置文件里面。

进入 xml 文件内部，首先增加参数标签

```
<param>
  <name>...</name>
  <key>...</key>
  <type>...</type>
</param>
```

增加前

```
<param>
  <name>Gray Code</name>
  <key>gray_code</key>
  <value>True</value>
  <type>bool</type>
  <option>
    <name>Yes</name>
    <key>True</key>
  </option>
  <option>
    <name>No</name>
    <key>False</key>
  </option>
</param>
```

增加后

而后更改输入输出的数据类型

```
<sink>
  <name>in</name>
  <type><!-- e.g. int, float, complex, byte, short, xxx_vector, ...--></type>
</sink>

<source>
  <name>out</name>
  <type><!-- e.g. int, float, complex, byte, short, xxx_vector, ...--></type>
</source>
```

更新前

```
<sink>
  <name>in</name>
  <type>complex</type>
</sink>

<source>
  <name>out</name>
  <type>byte</type>
</source>
```

更新后

3.3.5.在 grc 中安装 my_qpsk_demod

首先建立 build 文件夹，进入文件夹中运行指令 cmake ..

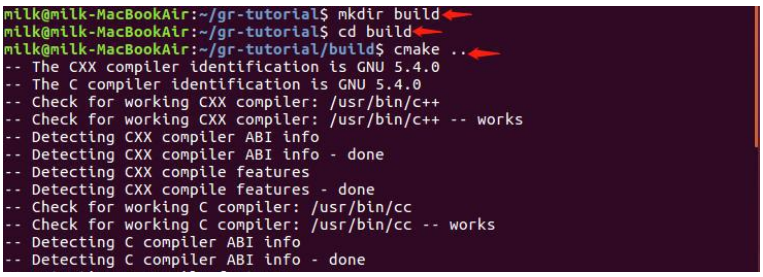


Fig 19. 使用 cmake ..构造编译步骤的配置文件。

然后运行指令 make，等待安装完毕，若中途出现错误，则需要回到原代码修改错误部分。

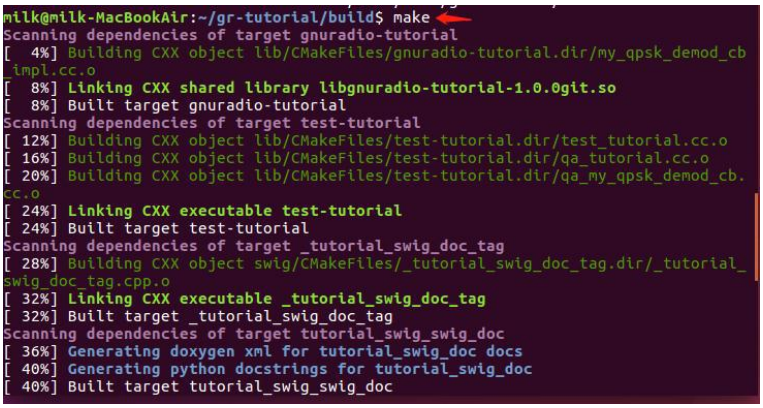


Fig 20. 使用 make -j8指令开始编译。

再运行指令 sudo make install

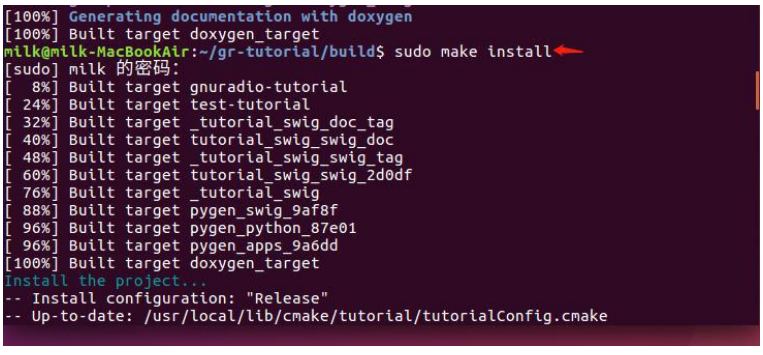


Fig 21. 使用 sudo make install指令将 gnuradio编译成库文件以供 python和 C语言调用。

最后输入 sudo ldconfig



Fig 22. 使用 sudo ldconfig指令使得 ubuntu系统识别出系统配置的变化。

至此，整个编写模块封装完毕。

打开 gnuradio-companion，可以在搜索中找到自己刚刚生成的模块，并制作流程检验其正确性，如下所示：

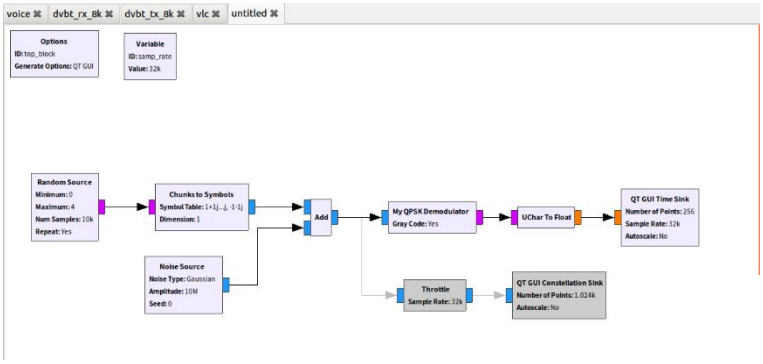


Fig 23. 新建模块搭建的系统。

点击运行按钮

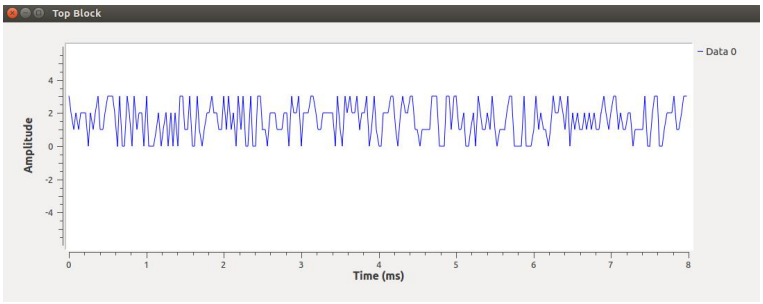


Fig 24. 新建模块搭建的系统运行效果。

3.4. 如何使用 C 语言在 gnu3.8.0 创建新模块

这个部分的参考在这个链接里面：使用 C 语言创建全新模块

使用的案例和上面的“3.3.如何使用 C 语言在 gnu3.7.13.5 创建新模块”是一样的，基本流程也都是一样的，这里就不再赘述太多，唯一的不同就是 gnu3.8.0 由于使用 yaml 语言写模块的界面，因此在 3.8.0 中需要写的文件是 yml 文件，而并不是 xml 文件。

4. 如何读懂原有模块的源代码并进行修改？

经过“二.2”和“二.3”的讲解，我们应当是已经找到了我们需要修改的.h 和.cc 文件了，下面一步就是开始阅读代码。

没有什么捷径，就是一步一步分析，读懂每一行在干什么，就抽丝剥茧地知道自己要改什么了。

这里选取我更改信道均衡模块的源代码作为案例，分析一下怎么更改原来模块中的源代码。

4.1. 问题分析

我们遇到的问题是，现在搭建的 MIMO OFDM 模型在接收端做信道均衡时，接收端的 QPSK 星座图映射在有噪声的情形下并没有出现“抖动”现象。经过初步的分析，问题应当是出在了接收端的“信道均衡模块”(OFDM Frame Equalizer Module)，这个模块不仅仅进行了信道估计和信道均衡，同时进行了最小距离分析并把含有噪声的点强行“拉回”星座图上的标准映射点。这个与我们一般意义下理解的“信道均衡”是不同的，一般不会“拉回去”。

所以，现在我们的任务就是找出与 equalizer 相关的.h 和.cc 文件，阅读源代码，并对源代码进行修改。

下面就从找出正确的.h 和.cc 文件开始下手，一步一步阅读相关的代码，最后重新编译得到调整后的模块，获得正确结果。

4.2. 寻找出多个.h 和.cc 文件并分析其间的关系

我们首先是要找到界面中的 OFDM Frame Equalizer 模块，看一下里面的参数是什么。OFDM Frame Equalizer 模块的截图如下所示：

我们可以看到，equalizer 填入的是 payload_equalizer 是一个变量，因此我们要去 all_success.grc 流程图文件的最上面，也就是定义 Variables 的区域去找这个 payload_equalizer 变量。

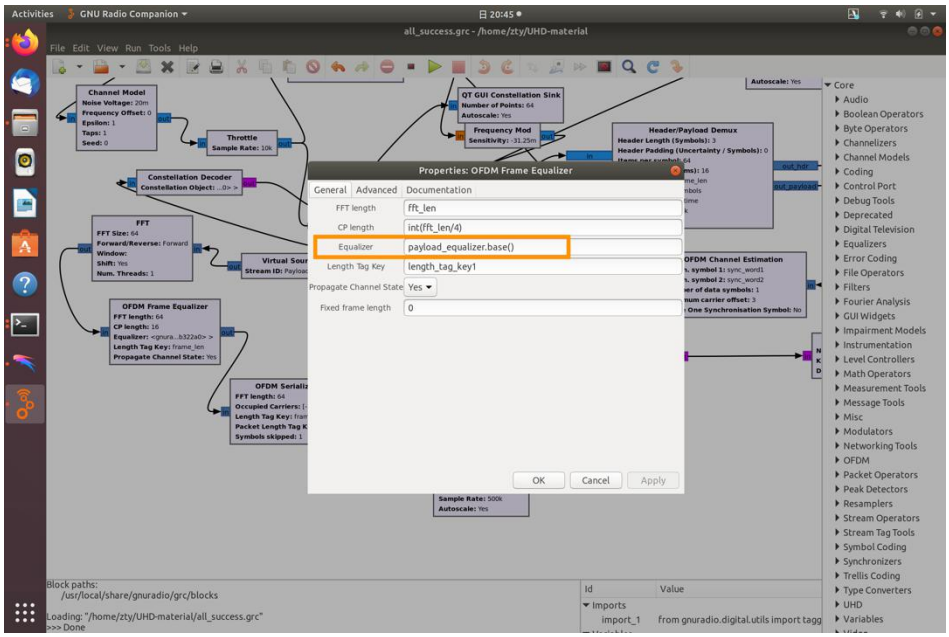


Fig 25. 找到 OFDM Frame Equalizer 模块的参数，发现 equalizer 这个参数填写的是 payload_equalizer.base()，这个 payload_equalizer 是一个变量 Variables，要去变量区找。

我们随后找到了定义 payload_equalizer 的那个 Variables 模块，如下图所示：

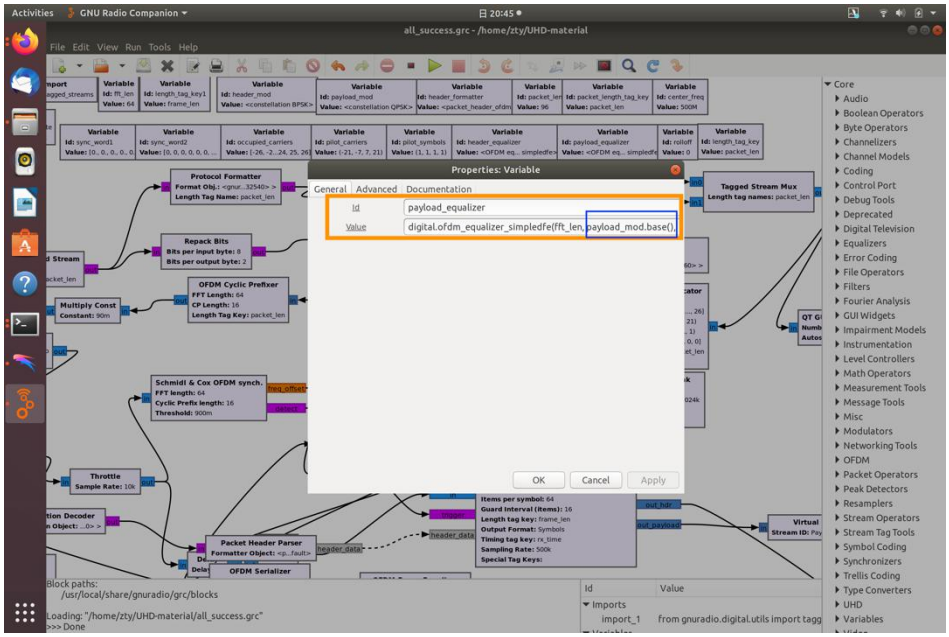


Fig 26. 找到定义了 payload_equalizer 变量的 Variables 模块（橙色框框出来了），里面我还画了一个蓝色框，这个之后再说。

在上图中，橙色框中，我们看到 payload_equalizer 本质是一个类的实体，这个类的名字叫做 ofdm_equalizer_simpledfe。根据我们 C++入门里面讲解的知识，

一般而言，一个类是要声明在和自己这个类同名的.h 文件里面的，所以我们进入 include 去查找可能和 ofdm_equalizer_simpledfe 相关的所有.h 文件，搜索结果如下 Fig 27.所示：

我们经过搜索，在/gnuradio/gr-digital/include/gnuradio/digital/文件夹下找到了所有与 equalizer 有关的.h 文件，并且仅仅选取命名中含有 ofdm 的文件，因为剩下的名字叫做 cma、lms、以及 kurtotic，这些与我们的 ofdm 一点关系也没有。

紧接着我们去/gnuradio/gr-digital/lib/文件夹下找到了所有上述.h 文件对应的.cc 源文件，如下图 Fig 28.所示。

至此，我们已经找出了所有与我们这个 equalizer 有关的.h 和.cc 文件了，下面我们深入进这些文件，进行代码阅读的工作。

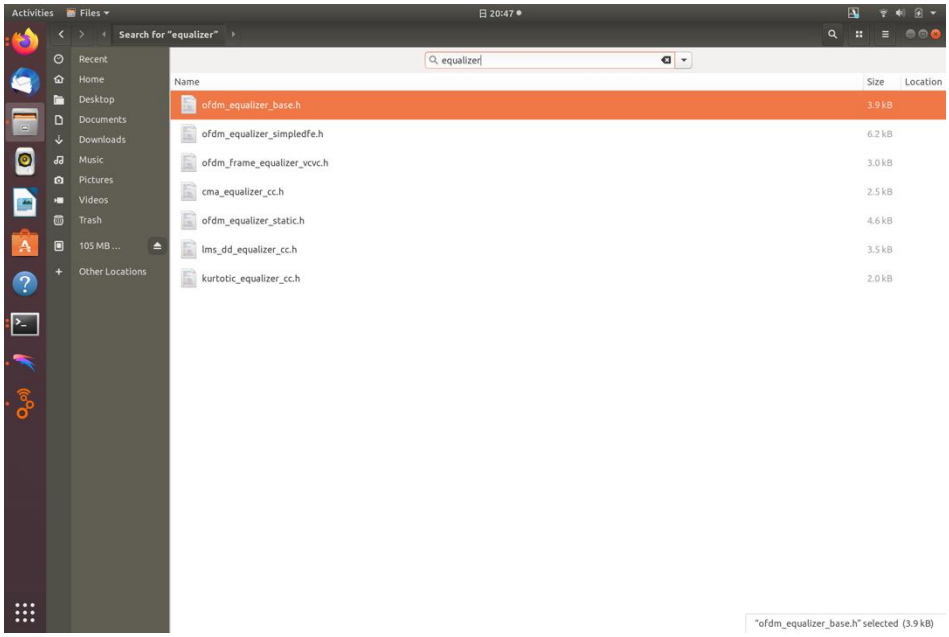


Fig 27. 从/gnuradio/gr-digital/include/gnuradio/digital/文件夹下找到了所有与 equalizer 有关的.h 文件，因为我们做的项目是 ofdm，因此我们只关心命名中含有 ofdm 的.h 文件，这是合理的，因为 cma、lms 以及 kurtotic 与我们的 ofdm 一点关系也没有。

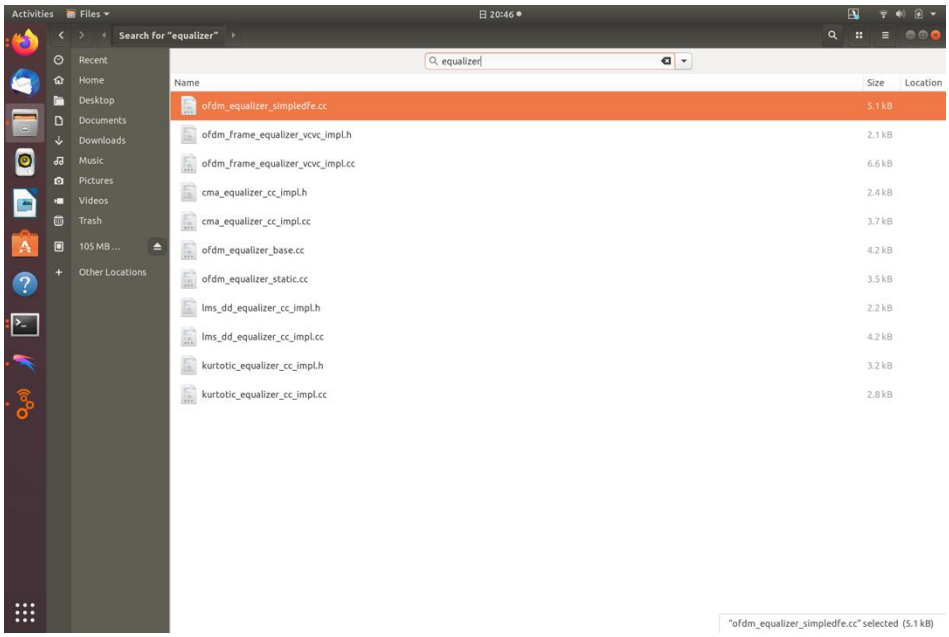


Fig 28. 从/gnuradio/gr-digital/ilib/文件夹下找到了所有与 equalizer 有关的.cc 文件，同样是仅仅选取命名中含有 ofdm 的文件。

4.3. 从顶层的.h 和.cc 文件开始直到最低的继承等级，一步一步阅读代码

在上一个步骤中我们已经得到了所有与 equalizer 有关的.h 和.cc 文件了，我们怎么开始着手阅读呢？

我们依然以界面为突破口，请大家回顾 Fig 23.中看到的 payload_equalizer 的类别为 ofdm_equalizer_simpledfe，我们就先看看 ofdm_equalizer_simpledfe.h，看一下这个文件里定义的这个类 ofdm_equalizer_simpledfe 是什么东西。

ofdm_equalizer_simpledfe.h 文件中的开头部分代码截图如下：

在下图 Fig 29.中显示，ofdm_equalizer_simpledfe 类继承自一个并不被包含在本文件中的 ofdm_equalizer_1d_pilots 类，因此我们下面的一步就是去找一下这个 ofdm_equalizer_1d_pilots 类在哪里，我们最终在 ofdm_equalizer_base.h 文件中找到了它，并发现 ofdm_equalizer_1d_pilots 类继承自更基本的 ofdm_equalizer_base 类，因此我们继续跟踪 ofdm_equalizer_base 类，如下截图 Fig 30.所示：



Fig 29. ofdm_equalizer_simpledfe.h 的开头部分，在橙色的框里面框出来的就是这个 ofdm_equalizer_simpledfe 类的声明，我们发现它是继承自 ofdm_equalizer_1d_pilots 这个类的。

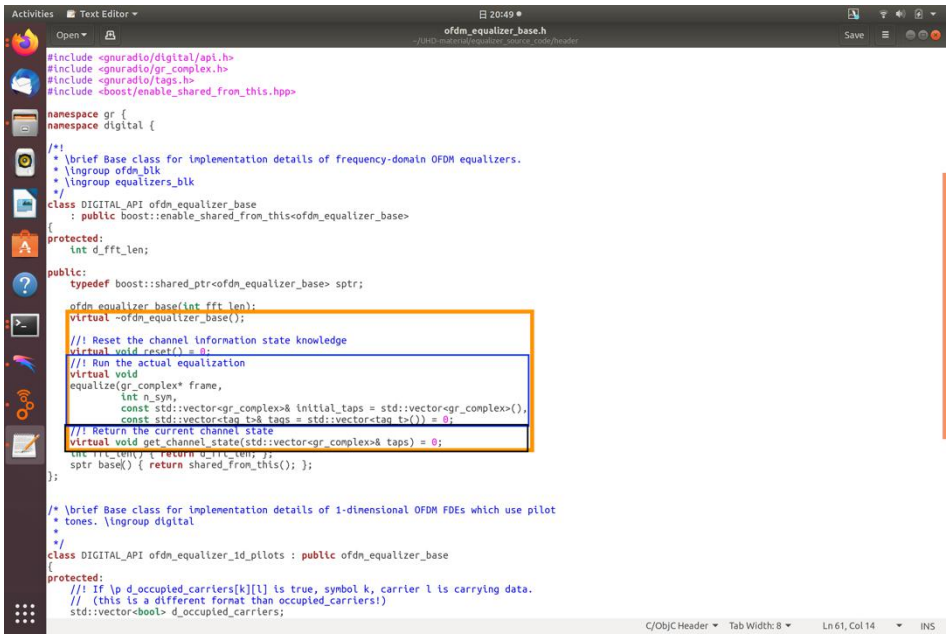


Fig 30. ofdm_equalizer_base.h 的 ofdm_equalizer_base 类被声明的部分。

我们看到，在声明中，橙色框框出了四个纯虚函数（就是根本没有实现的函数），也就是说，ofdm_equalizer_base 的子子孙孙就是要去实现这四个纯虚函数的。

这四个纯虚函数中，蓝色框里的是 equalize 函数，用于做信道均衡；黑色框框出的是 get_channel_state 函数，用于做信道估计。

这样我们就很明朗了，我们再回头看看 ofdm_equalizer_base 的儿子 ofdm_equalizer_1d_pilot，我们发现这个儿子类实现了 get_channel_state。但是我们不关心，我们继续去看 ofdm_equalizer_1d_pilot 的儿子（也就是 ofdm_equalizer_base 的孙子）——也即是我们魂牵梦萦的 ofdm_equalizer_simpledfe 类，我们发现这个孙子就是我们找的，就是这个孙子实现了 equalize 函数！爽了！

于是我们可以打开 ofdm_equalizer_simpledfe.cc 看看这个 equalize 错在哪了，如下图 Fig 31.所示。

【我们现在可以对 grc 界面大声说一句：哦，原来就是你小子把搭建的兄弟们引到这儿来的啊！咱们清华-北交平台小组托我给 ofdm_equalizer 带个话，你这个 constellation 不抖动问题已经被我们包围了，只要你快速暴露自己的 bug 在哪，保证立马干掉，重新编译，让你这个 bug 永无还手之力！】

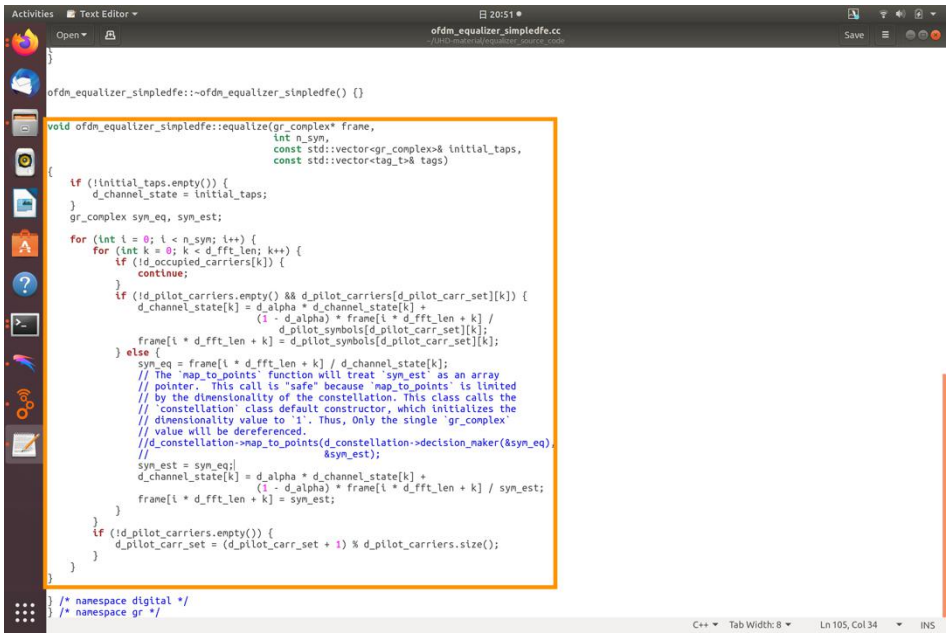


Fig 31. ofdm_equalizer_simpledfe.cc 的 equalize 函数源代码。

好的，现在就是革命成功的前夜！宜将剩勇追穷寇，不可沽名学霸王！我们穷追猛打这个 equalize 函数，在下一个部分中，我将会一句一句分析这个 equalize 函数的每句话都是什么意思。

4.4. 找到问题修改并编译重置

如下代码段所示是我把它自己原来的英文注释全部按照我的理解改成的汉字注释，这样就清楚很多了。

```
void ofdm_equalizer_simpledfe::equalize(gr_complex* frame,
                                        int n_sym,
                                        const std::vector<gr_complex>& initi
al_taps,
                                        const std::vector<tag_t>& tags)
/*首先介绍参数
 *参数一共是 4 个
 *第一个参数的类型是复数序列，经过查证，是界面上 OFDM Frame Equalizer 的输入 input
 *第二个参数的类型是整数，经过查证，这个是 64，也就是导频的个数
 *第三个参数的类型是复数数组，经过查证，是初步估计出来的信道状态情况
 *最后一个参数的类型是标记数组，初始化为空数组
 */
{
    if (!initial_taps.empty()) {
        //首先，如果曾经有初步估计出来的信道模型，那就把现在的信道模型先置为老模型
        //（这个模型之后会被更新）
        d_channel_state = initial_taps;
    }
    //定义两个用于中转的复数
    gr_complex sym_eq, sym_est;

    //对于每一个 symbol
    for (int i = 0; i < n_sym; i++) {
        //对于每一个子载波
        for (int k = 0; k < d_fft_len; k++) {
            //如果这个子载波没有数据，直接跳过
            if (!d_occupied_carriers[k]) {
                continue;
            }
            //如果有数据，那就开始做下面的操作
            //如果导频不是空数组，并且这个导频针对的是传了数据的载波
            if (!d_pilot_carriers.empty() && d_pilot_carriers[d_pilot_carr_s
et][k]) {
```

```
                //那么就更新信道模型（就是说这个子载波传了导频，使用这个导频更新信道模型，这就是对原先估计的信道模型的优化）
                d_channel_state[k] = d_alpha * d_channel_state[k] +
                                     (1 - d_alpha) * frame[i * d_fft_len + k] /
                                         d_pilot_symbols[d_pilot_carr_set][k];
                //更新过信道模型之后，做信道均衡，得到均衡出的信号，把输入的序列对应位置的值就改成信道均衡后的值
                frame[i * d_fft_len + k] = d_pilot_symbols[d_pilot_carr_set][k];
            }
            //如果导频是空数组，或者没有有效的导频
            else {
                //那肯定是没法对原先估计出的信道进行优化了
                //所以就直接基于原先估计过的信道模型给出信道均衡的值 sym_eq 作为中转
                sym_eq = frame[i * d_fft_len + k] / d_channel_state[k];
                //！！！！！！！d_constellation 这一行就是问题所在！！！！！！！
                //这个 gnuradio 编写的人认为，既然没有通过导频对信道估计进行优化，那进行的信道均衡肯定是不准的
                //于是，他用 d_constellation 里面的 decision_maker 方法，把这个 e 没有估计准确的点强行
                //通过基于最近欧几里得距离的判决域方法，强行把这个有噪声的估计值给“拉回”星座图上的标准点！
                //这就是为什么星座图不抖动了！
                //既然明白这个原理了，于是我们可以把这一句直接改成下面第三行一行的句子
                d_constellation->map_to_points(d_constellation->decision_maker(&sym_eq), &sym_est);
                //上面这句话应该改成下面这句注释中的话！
                //sym_est = sym_eq;

                //接下来就是用这个均衡过的值再次优化信道模型
                d_channel_state[k] = d_alpha * d_channel_state[k] +
                                     (1 - d_alpha) * frame[i * d_fft_len + k] / sym_est;
                //把信道均衡后的值赋给序列，就做完了均衡
                frame[i * d_fft_len + k] = sym_est;
            }
        }
        //如果导频序列是空的，那就初始化一下
        if (!d_pilot_carriers.empty()) {
```

```
            d_pilot_carr_set = (d_pilot_carr_set + 1) % d_pilot_carriers.siz
e();
        }
    }
}
```

至于上面代码段中提到的 constellation 里面的 decision_maker 函数以及 map_to_points 函数在哪里怎么找，仍然遵循我曾经讲过的原则，我们看 Fig 23. 的图例里面我卖了一个关子，Fig 26.中用蓝色框框出来的东西是 payload_mod.base()，于是我意识到，这个也是流程图里定义的一个 Variables，于是我就去找定义这个变量的模块，如下图所示：

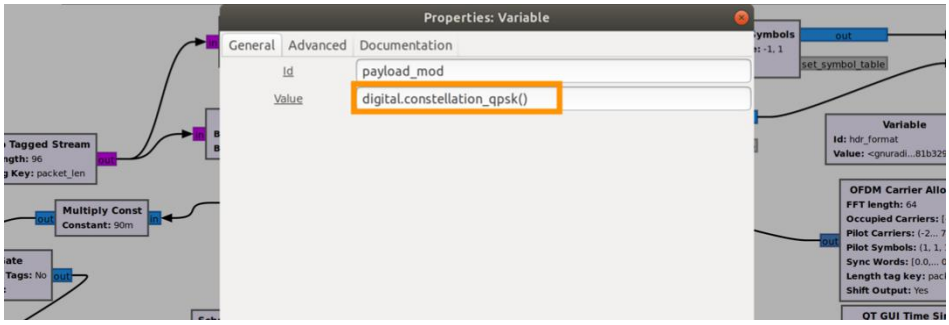


Fig 32. payload_mod 的定义，我们发现它是一个 constellation_qpsk 类型，于是我们去找和 constellation 有关的.h 和.cc 文件。

我在 constellation.cc 文件中最终发现了这个类的定义，如下图 Fig 33.所示：

至此，错误已经改正，问题也全部纠正完毕，我们重新执行 cmake、make、install 等的语句，编译成功之后，就得到了抖动的星座图。

需要注意的是，这里的 cmake 和 make 的执行方法是：

首先进入安装的时候的 gnuradio 文件夹下，之后进入 build 文件夹下，在这个文件夹下开启 terminal

之后在 terminal 中输入 cmake ..再输入 make -j8 之类的就行了。

已经尝试过别的位置，但是均证明失败，最好还是遵循上述过程。

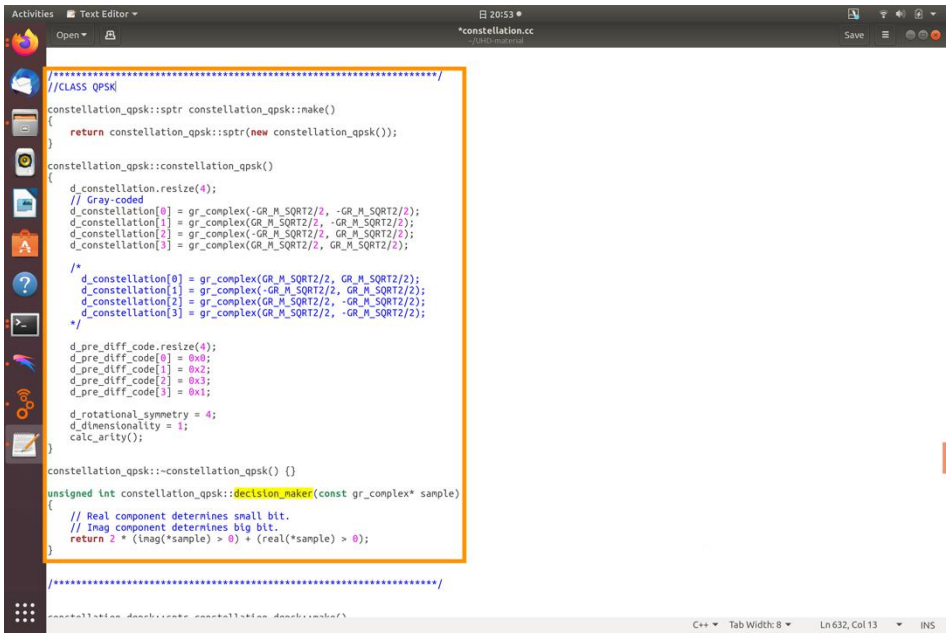
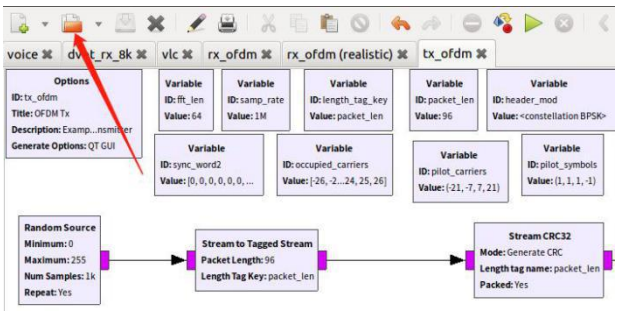


Fig 33. constellation_qpsk 类别的定义，我们可以看到其中的 decision_maker 就是罪魁祸首了。

四、 Gnuradio 平台 OFDM 相关模块代码实现原理（本部分直接复制学长们之前撰写的文本）

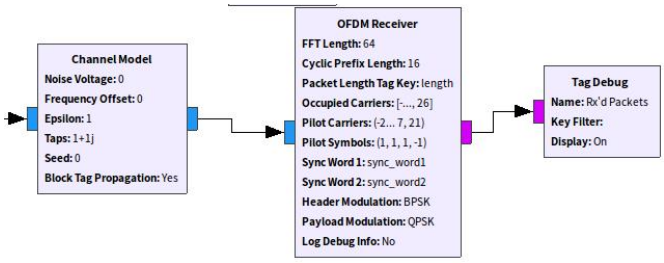
在 Ubuntu 系统中，打开“文件”，进入“home/gnuradio/gr-digital/examples/ofdm”文件夹，选中“tx_ofdm.grc”和“rx_ofdm.grc”文件，将其复制到/home 文件夹下。

打开终端，运行“gnuradio-companion”进入 gnuradio 图形界面，选择“open an existing flow graph”。如下图：

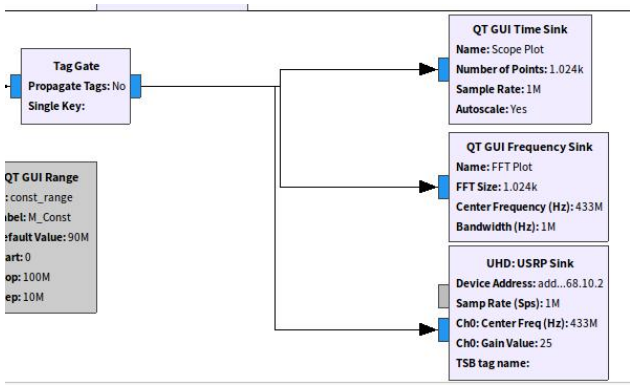


1. 发送端步骤

将打开的“tx_ofdm.grc”中的发送信号到模拟接收端的模块替换为频域和时域波形显示模块和“UHD:USRP Sink”模块，如下所示：

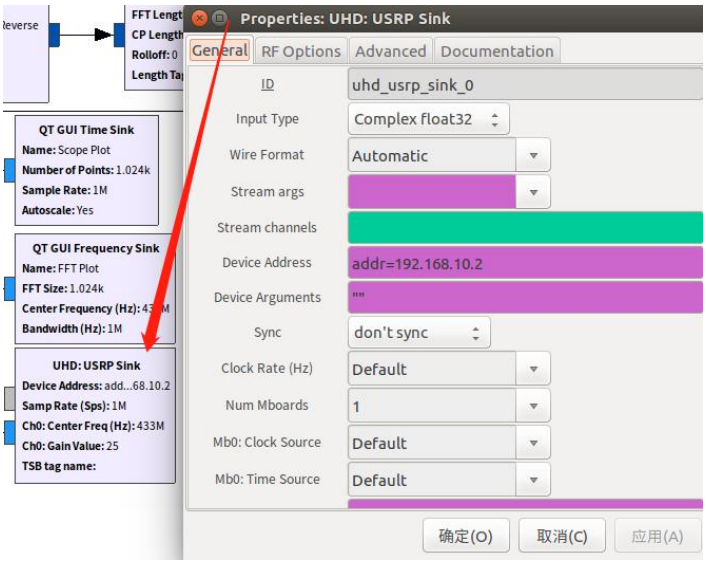


替换前

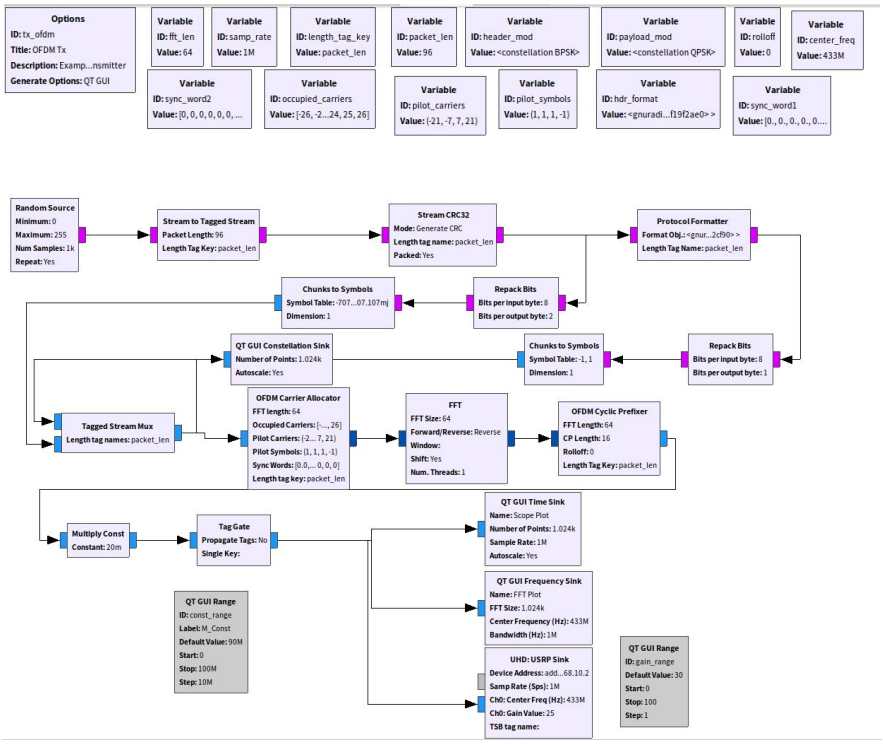


替换后

其中 UHD:USRP Sink 模块上调整设备地址、采样率、中心频率和增益等变量值，配置如下



然后添加相应的变量模块，最后经整理得到整体流程框图如下：



2. 发送端模块分析

Random Source：实现的功能是随机产生数据样本，范围 0-255，数量 1k，重复发送，输出数据类型为字节。

输入变量：

Minimum：最小值

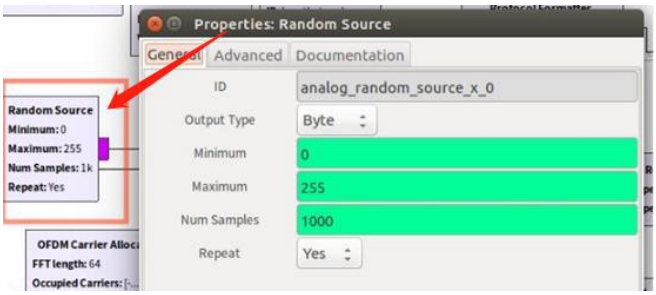
Maximum：最大值

Num Samples：样本数量

可选变量：

是否循环：Yes。

输出：数据类型 byte。



Stream to Tagged stream: 该模块实现的功能是将输入随机数据流进行打包，形成一定长度的帧数据流，并给数据贴上标签。该模块按设定长度输出打包好的帧数据流，所以数据帧长度为 96 字节。

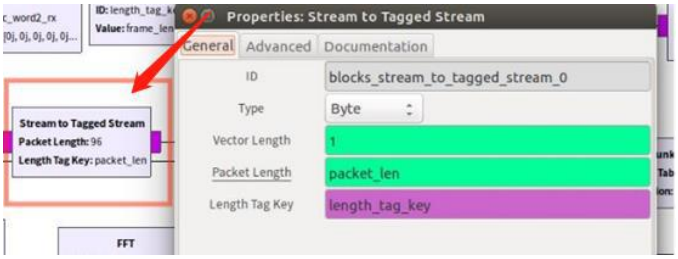
输入变量：

Vector length ： 1，输入向量的个数，默认为 1 个；

packet length：packet_len，定义一帧数据的长度，在流程图的变量模块中 packet_len 设定为 96；

Length Tag Key: length_tag_key，给输出数据打上标签，其标签名为 length_tag_key，值为 96.

输出：带标签的帧数据流，数据类型 byte.



Stream CRC32: 该模块实现的功能是对输入的每帧数据形成校验位，并将形成的 4 个字节的校验字附在每帧数据的末尾。经过该模块后输出的一帧数据长度较输入之前增加了 4 个字节，所以一帧数据长度变成 100 个字节，共 100*8 个比特。

输入变量：

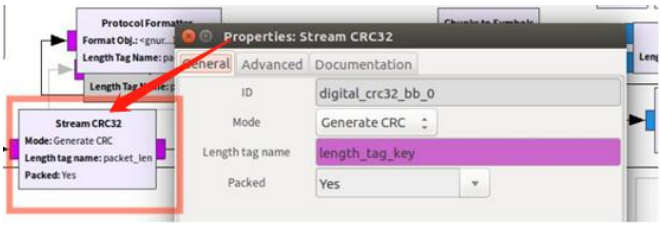
Length tag name ：length_tag_key，输入的标签，值为数据帧长度，该模块需要知道一帧数据的长度是多少；

可选变量：

Mode: Generate CRC。这里是发送端，所以选产生 CRC 校验码，而不是校验接收到的数据。

Packed: Yes。 告诉模块输入的数据是否是打包好的数据。

输出：带标签的帧数据流（含校验位），数据类型 byte.



Protocol Formatter: 该模块根据接收数据帧中信息的创建一个帧头，通常用于 MAC 级处理。帧头(标头)的生成格式是基于指定的对象，该对象是从 header_format_base 类派生的对象。指定帧头共 6 个字节 6*8 个比特，其中字节 1 代表一帧数据的长度，字节 2 和 3 一起表示当前帧的帧号，其它三个字节还不知道具体表示什么。

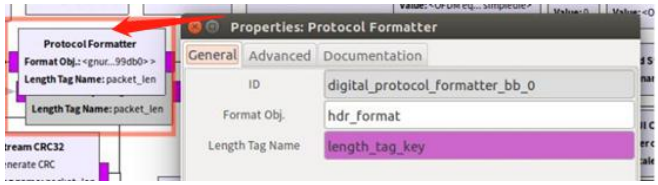
输入变量：

Formatter Object ：生成帧头时要使用的格式对象，指定形成的帧头格式；

Length Tag Name: "packet_len". 标签，值为帧长度。该模块在对数据进行处理时需要知道一帧数据的长度。

输出：

帧头数据流，数据类型 byte.



Repack Bits: 该模块实现的功能是将 8 比特一组(字节)的数据变成 2（QPSK）或者 1（BPSK）比特一组，由最低位到最高位依次转换。因此，在解码的时候需要特别注意每个比特在一个字节中的位置。

输入变量：

Bits per input byte: 输入端口每个字节中包含的比特数，指定为 8；

Bits per output byte: 输出比特中的字节数；（输出端口每个字节中所包含的比特数，payload_mod.bits_per_symbol 中的 payload_mod 在流图的变量模块中设定为 QPSK，因此每个符号应包含 2 个比特数）。

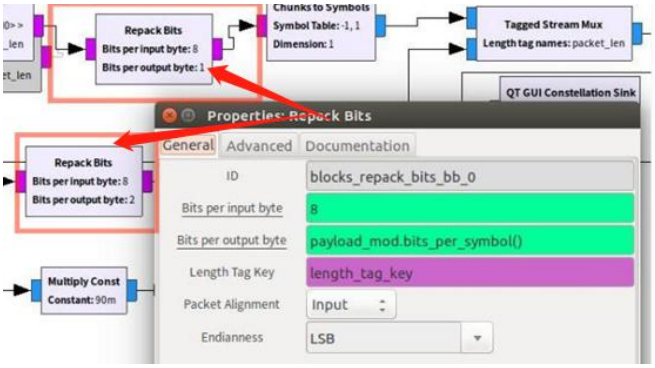
Length Tag Key: length_tag_key. 标签。该模块在对数据进行处理时需要知道帧长。

可选变量：

Packet Alignment: input. 选择需要进行该操作的端口，这里只有一个 input 端口可选。

Endianness: LSB. 从一个字节的最低位开始操作。

输出：带标签的帧数据流，数据类型 byte，但每个 byte 包含 2 或者 1 个字节.



Chunks to Symbols: 符号映射模块，完成数据块到符号流的转换。可选的符号参数有-1+0j、1+0j 和 0.707+0.707j.....-0.707-0.707j 两组，因此该模块的功能分别是实现 BPSK 映射和 QPSK。

输入变量：

Symbol Table ： header_mod.points(), 该值在变量模块中已定义，其映射规则是输入 0 时输出-1+0j，输入 1 时输出 1+0j；payload_mod.points(), 该值在变量模块中已定义，两比特位输入对应的输出映射，(11,10,00,01)对应的映射值有(0.707+0.707j, -0.707+0.707j, -0.707-0.707j, 0.707-0.707j)；

Dimension : 1， 整个流图始终处理的都是一维数据，所以这里同样是 1 维。

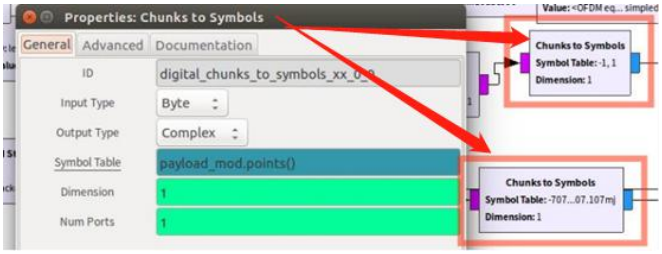
Num Ports: 1. 端口数，同样这里选 1。

可选变量：

Input Type: Byte: 输入数据类型需要与前一模块的数据类型相同。

Output Type: Complex. 输出数据类型为复数。

输出：带标签的帧数据流，数据类型 complex。



Tagged Stream Mux: 将 2 个流作为输入。数据包从每个输入流依次输出。输出信号有一个新的长度标签，旧的长度标记将被丢弃。这里是要将两路信号（帧头和有效载荷）合并，因此需要增加数据帧的长度。之前我们分析过数据帧长为 100 个字节共 100*8 个比特，经 QPSK 映射后为 100*4 个符号；同样针头有 6 个字节共 48 个比特，经 BPSK 映射后变成 48 个符号。因此，将这两路信号进行合并后得到的一帧数据长度为 100*4+48=448 个符号。

输入变量：

Number of Input ： 2，输入端口数，有两路信号输入，因此这里应填 2。

Length tag names : length_tag_key， 输入数据帧的长度， 96。

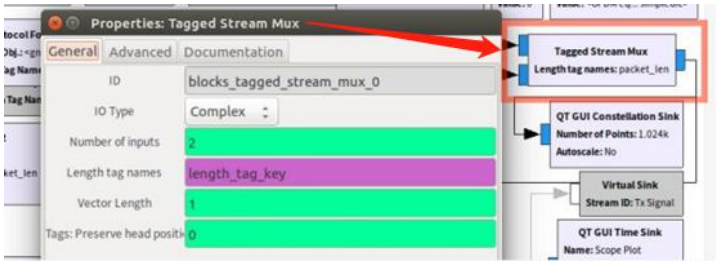
Vector Length : 1. 和信源保持一致；

Tags: Preserve head position on input: 保留此输入端口上标记的头部位置；

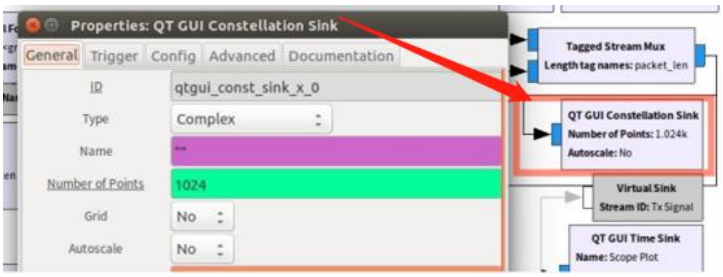
可选变量：

IO Type: complex，端口输入数据类型。

输出：带标签的帧数据流，数据类型 complex。



QT GUI Constellation Sink 展示信号的星座映射，图示。



OFDM Carrier Allocator: OFDM 子载波分配模块，也即串并转换模块。该模块的作用是给每个子载波分配相应的值，数据相应地实现串并转换。这里设总载波个数为 64，载波类型包括以下几种：数据子载波(Occupied carriers)、导频子载波(Pilot carriers)、虚拟子载波(Virtual carriers, VC)和直流子载波(DC carrier)。其中数据子载波可以用来携带数据和同步字，导频载波用于发射特定符号数据，空载波的值默认为 0。在试验中发送端 OFDM 子载波的编号规则是：为了保持左右对称，将 64 个子载波编号为[-32,31]。其中，

最左边 6 个子载波[-32,-27]和最右边的 5 个子载波[26,31] 称为虚拟载波，即不使用的载波它的值默认为 0，其作用是为了减少带外功率和抑制 ACI；

编号为[-26,-22]、[-20,-8]、[-6,-1]、[1,6]、[8,20]、[22,26]都是数据子载波；

编号为-21、-7、7、21 为导频子载波；

编号为 0 为直流子载波，作用是降低 OFDM 信号的峰值信噪比以及硬件电路的复杂度；

由此可知，64 个子载波中有效子载波只有 48 个。这样我们便可以计算出承载一帧有效数据（包括帧头和负载共 448 个符号）所需要的 OFDM 符号数（将 64 个子载波组成的并行数据称为一个 OFDM 符号），即 448/48=9.33 个 OFDM 符号，也就是说传输一帧有效数据需要 10 个 OFDM 符号。另外，为了接收端解调方便，本实验中的 OFDM 信号需要加入 2 个符号的同步字置于一帧有效数据的开头。所以，最后传输的一帧数据中包含了 12 个 OFDM 字符，2 个同步字和 10 个有效数据。

输入变量：

FFT length ： fft_len, 傅里叶变换的长度，fft_len 在流图变量模块中指定为 64。

Occupied Carriers: occupied_carrieds， 数据子载波编号，occupied_carrieds 在流图变量模块中指定为[-26,-22]、[-20,-8]、[-6,-1]、[1,6]、[8,20]、[22,26]。

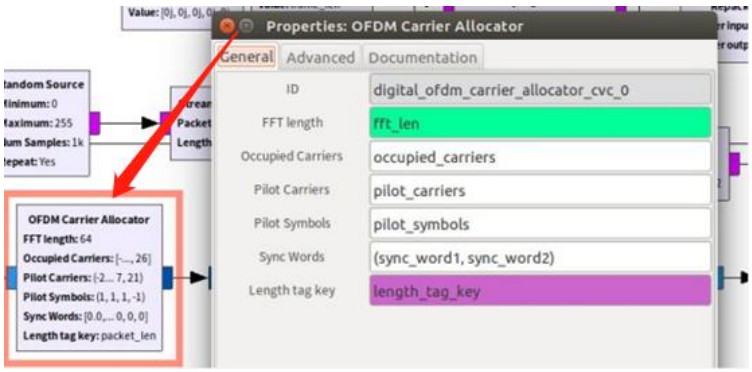
Pilot Carriers: pilot_carrieds， 导频子载波编号，pilot_carrieds 在流图变量模块中指定为-21、-7、7、21。

Pilot Symbols: pilot symbols, 导频子载波上对应的符号（值），pilot symbols 在流图变量模块中对应上述载波编号指定为：1，1，1， -1。

Sync_words: (sync_word1, sync_word2)，同步字信息，sync_word1, sync_word2 是在流图变量模块中指定信息。其中同步数组 1 的值为[0., 0., 0., 0., 0., 0., 0., 1.41421356, 0., -1.41421356, 0., 1.41421356, 0., -1.41421356, 0., -1.41421356, 0., -1.41421356, 0., 1.41421356, 0., -1.41421356, 0., 1.41421356, 0., -1.41421356, 0., -1.41421356, 0., -1.41421356, 0., -1.41421356, 0., 1.41421356, 0., -1.41421356, 0., 1.41421356, 0., 1.41421356, 0., 1.41421356, 0., -1.41421356, 0., 1.41421356, 0., 1.41421356, 0., 1.41421356, 0., -1.41421356, 0., 1.41421356, 0., 1.41421356, 0., 1.41421356, 0., 0., 0., 0., 0., 0.]；同步数组 2 的数据为[0, 0, 0, 0, 0, 0, -1, -1, -1, -1, 1, 1, -1, -1, -1, 1, -1, 1, 1, 1, 1, 1, -1, -1, -1, -1, -1, 1, -1, -1, 1, -1, 0, 1, -1, 1, 1, 1, -1, 1, 1, 1, -1, 1, 1, 1, 1, -1, 1, -1, -1, -1, 1, -1, 1, -1, -1, -1, -1, 0, 0, 0, 0, 0]。

可选变量：无

输出：OFDM 符号流，数据类型 complex。



FFT：傅里叶变换模块。对已分配好的子载波符号进行傅里叶变换，完成 OFDM 调制。

输入变量：

FFT Size ： fft_len，傅里叶变换的长度，fft_len 在流图变量模块中指定为 64。

Windows：傅里叶变换的窗函数，不清楚()代表什么。

Num. Threads: 1，线程数设为 1，C++中多线程数在任务量大时才有效，当任务量小时采用多线程会消耗额外的通信开销而导致其效率比单线程还低。因此，这里 64 点 DFT 采用单线程较好。

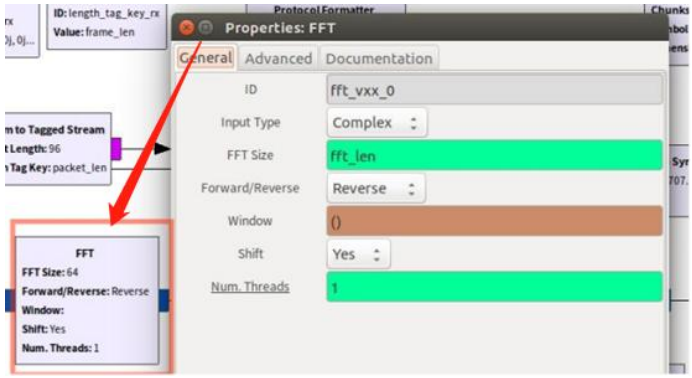
可选变量：

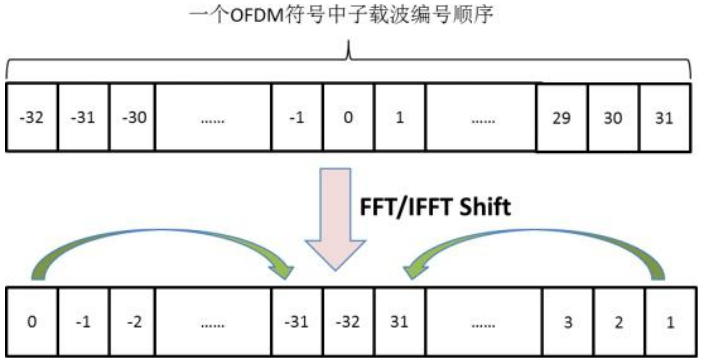
Input Type: Complex，输入的数据类型为复数。

Forward/Reverse: Reverse, FFT 变换类型，Forward 表示 FFT 变换，Reverse 表示 IFFT（逆傅里叶）变换。

Shift：shift，傅里叶变换后是否将频率点（下标）移动，shift 表示需要移动，移动规则如下图所示。这里需要注意的是在接收端 FFT 模块中需要做同样的设置，否则无法正常解调信号。

输出：OFDM 符号流，数据类型 complex。





OFDM Cyclic Prefixes: 给 OFDM 符号添加循环前缀。OFDM 信号中加循环前缀的目的是为了防止 OFDM 符号间干扰（ISI）。不清楚为什么选择 1/4 符号长度。

输入变量

FFT Length ： fft_len，傅里叶变换的长度，64。

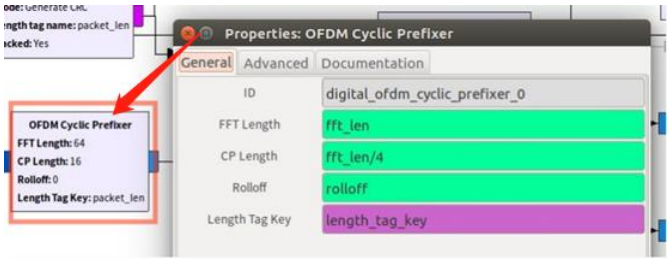
CP Length ： fft_len/4，循环前缀的长度，其值为 fft_len/4=16。

Rolloff: rolloff，成形滤波器参数设置，rolloff 为升余弦滚降。不清楚具体作用，在频域波形图上的直观表现是边缘下降沿坡度。

Length Tag Key : length_tag_key，数据帧的长度，同之前模块中的值一样。

可选变量：无。

输出：OFDM 符号流，数据类型 complex。



Multiply Const: 衰减器模块。该模块通常将输入的数据乘上一个常数，从而达到改变输入信号幅度的目的。在这里为了防止调制好的 OFDM 信号在进入功率放大器后产生非线性失真，需要将输入信号乘上一个小于 1 的常数，解调不出数据包的时候可能是因为接收信号失真严重，可以考虑进一步调小常数。

输入变量：

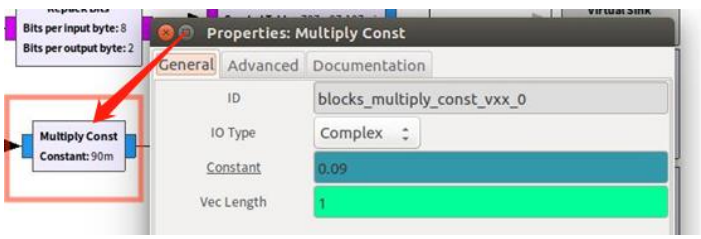
Constant ： M_const，一个小于 1 的浮点型常数。

Vec Length: 1.。

可选变量：

IO Type : complex。

输出：OFDM 符号流，数据类型 complex。



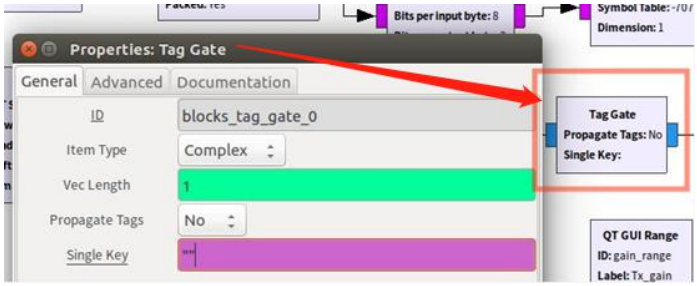
Tag Gate: 控制标记传播。使用此块可阻止标记传播。

输入变量：

Single Key: 要停止的标记的键。不清楚""的意思。

可选变量：

Propagate tags：当此设置为 true 时允许标记通过此块。



UHD: USRP Sink: 信号发送模块。该模块实现信号从软件到硬件转换的桥梁作用，将信号送到硬件再经射频天线将信号发送出去。本实验中所用的硬件主要的任务是完成插值、上变频、D/A 转换、功放、混频、最后经射频天线发射出去。

输入变量：

Device Address ： usrp_addr， 输入 USRP 设备的地址，通常一台设备对应一个 IP 地址，其储存在芯片 EPROM 中，主机通过寻找该地址来实现通信。usrp_addr 是流图变量模块中的一个变量名，其值为“addr = 192.168.10.2”。另外，要想完成主机和设备通信还需要对主机进行设置。

Samp Rate (Sps): samp_rate, 采样率。通常可以理解为主机和设备之间传输数据速率，最大为 50MS/s(8bit)或 25MS/s(16 bit)。samp_rate 的值在流图的变量模块中设为 1MHz。

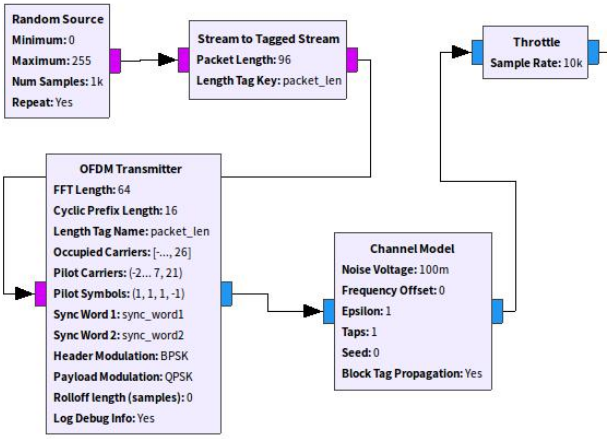
Center Freq (Hz): c_freq, 中心频率。指信号在无线环境中传播的频率，也即射频。c_freq 的值在流图的变量模块中设为 433MHz。

Gain Value: Tx_gain， USRP 发送增益，单位是 dB。Tx_gain 的值在流图的变量模块中设为 25dB。

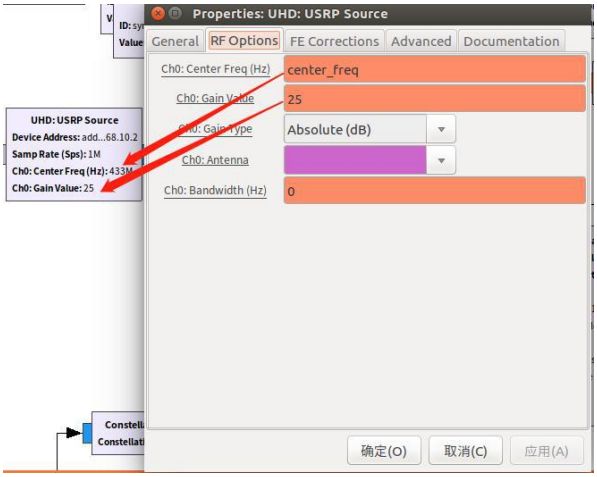
输出：射频信号，数据类型：模拟信号。

3. 接收端步骤

将打开的“rx_ofdm.grc”中的接收模块替换为“UHD:USRP Source”，如下所示：

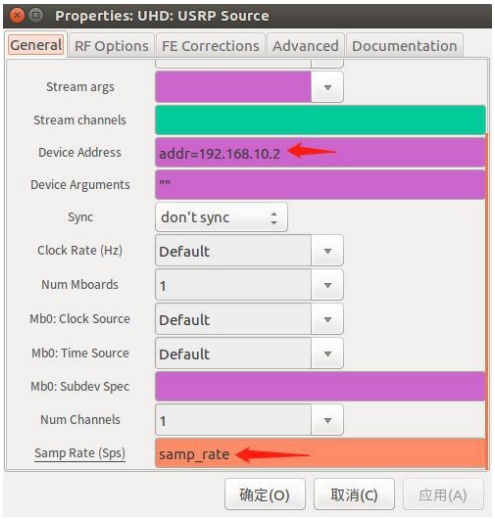


替换前

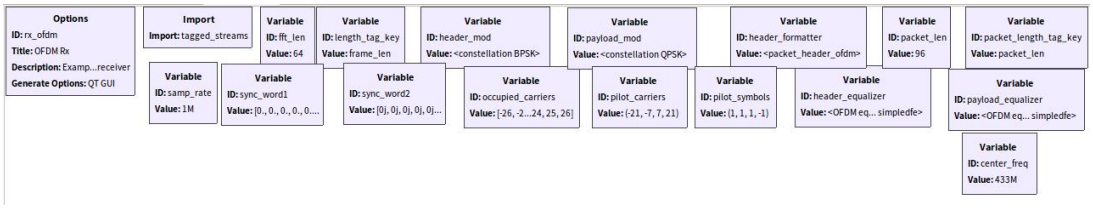


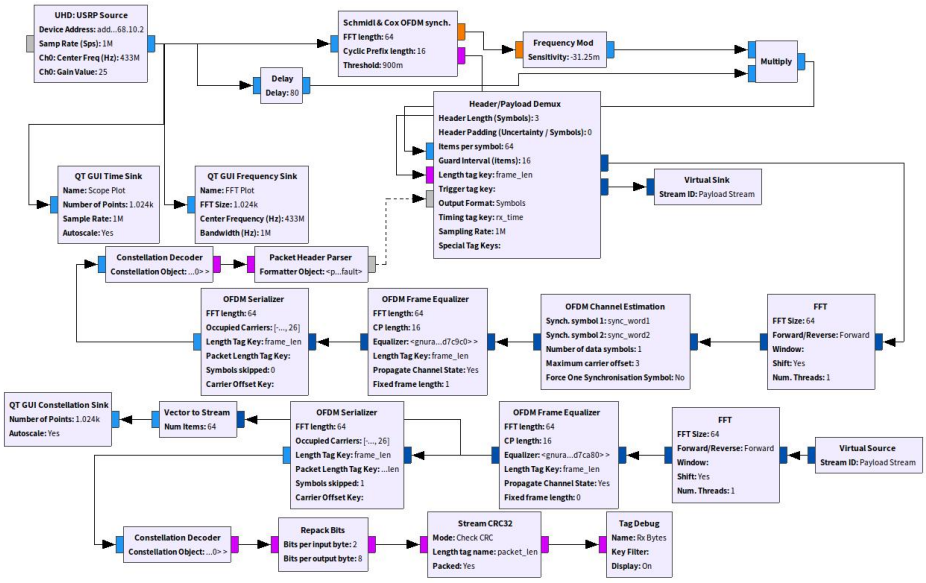
替换后

且将中心频率的值设为 center_freq，把增益值调整为 25，而后需要更改地址与采样速率，增加变量模块。



最后将整体流程进行重新整理，使其美观易读。最终收端呈现效果如下：





4. 接收端模块分析

UHD USRP Source：该模块可以实现硬件和软件之间的连接。具体是将硬件 USRP N210 接收到的信号（经过一系列处理后得到的信号）传送到主机。需要设定的参数是 USRP 设备地址，采样频率以及中心频率。输出是复数类型的 I/Q 数据流

Schmidl & Cox OFDM synch：该模块实现两个功能，第一实现符号定时同步；第二去除小数部分的载波频偏。该模块所使用的方法是根据文献【1】编写的，但是整数倍频偏纠正的方法没在该模块中给出。

输入变量：

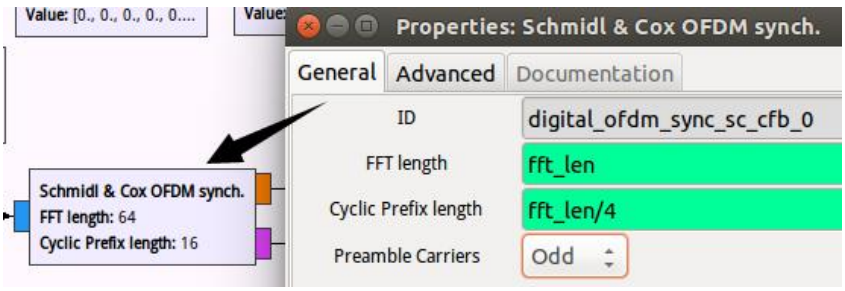
FFT length：fft_len。其中 fft_len 是傅里叶变换的长度，也是指 OFDM 符号中子载波的总个数。fft_len 在流图的变量模块中设定为 64；

Cyclic Prefix length： fft_len/4。循环前缀的长度，值为傅里叶变换长度的四分之一。

可选变量：

Preamble Carriers：Odd。模块使用同步字里面的奇数载波来计算起始点和载波频偏。使用奇数载波的目的是什么呢？在进行定时和载波频偏纠正时考虑的是第一同步字,而第一个同步字在频域中偶数载波上有值，而奇数载波为了零，但在时域是对称的。可以肯定的是奇数载波为 0 是同步字 1 的特征,因为所有的 symbol 在奇数载波上都是有值的，但并没有表明定时和去载波频偏算法中和奇数载波的关系。

输出：输出两个值，一个是载波频偏（小数倍），一个是定时信息。小数倍的载波频偏是经过归一化的：$\varepsilon = 2.0 \times \frac{\text{output}}{\text{fft_len}}$，可以得到输出 output 的变化范围为$[-\pi,\pi]$，也就是说载波频偏如果超过该范围便无法纠正。（其实指的是相位的变化，这是因为在数字域里面频率都被奈奎斯特采样率给归一化了,也就是说模拟频率和数字频率存在一定映射关系，而这种映射关系往往体现在相位的变化上。通常数字域中相位使用得多，模拟域中频率使用得多）。定时信息是一串和数据等长的 0,1 序列,起始点对应的地方设为 1，其它为 0。



Delay： 延迟模块。实现数据流指定长度延时。这里该模块的作用是使输入数据延迟一个 OFDM 符号。

输入变量：

Delay：fft_len+fft_len/4。延迟参数，当该值为正时输出在数据前端添加相应个数的 0，当该值为负时输出舍弃相应个数的数据。fft_len 上面已经介绍其值为 64，加上循环前缀的长度为 80，刚好一个 OFDM 符号的长度。

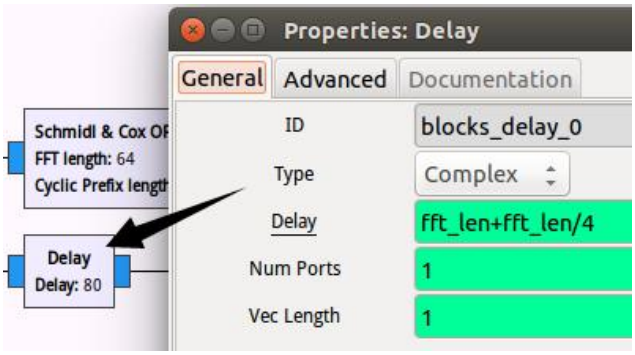
Num Ports: 1，输入端口数，1 个。

Vec Length: 1，输入向量数，1 个。

可选变量：

Type：Complex。选择输入数据类型。

输出：延迟一个 OFDM 符号后的信号流，数据类型复数。



Frequency Mod： 频率调制模块。作用是产生一个和载波频偏对应的序列，进而将该序列和接收的数据流相乘，达到去除载波频偏的目的。

输入变量：

Sensitivity: -2.0/fft_len。其中 fft_len 的值是 64。

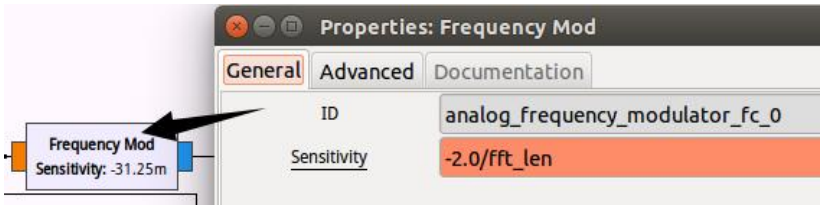
可选变量：无。

输出：经过该模块后输出的序列 x(n)为：

$$x(n) = e^{-j\frac{2\times output\times n}{fft_len}} = e^{j\times sensitivity\times output\times n}, n = 1,2,3,\ldots$$

* MERGEFORMAT (1)

其中，output 是模块 Delay 输出的载波频偏，n 的值和输入信号长度一样。



Multiply： 该模块实现两个向量相乘。作用是去除小数倍载波频偏，消除 ACI 干扰。

输入变量：

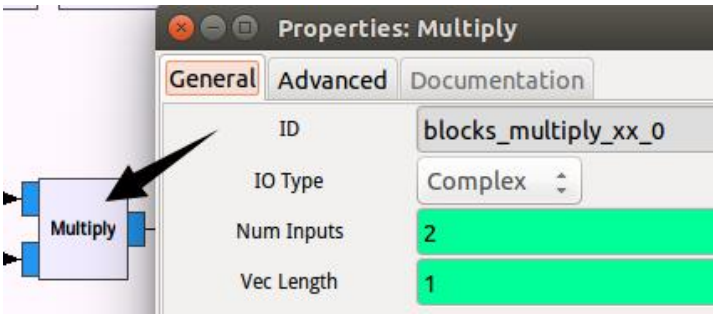
Num Inputs：2。 输入端口数为两个。

Vec Length: 1。向量长度为 1。

可选变量：

IO Type：Complex。输入和输出端口的数据类型为复数。

输出：去除小数倍载波频偏的数据流，数据类型为复数。



Header/Payload Demux： 该模块的作用是根据定时信息和帧头信息，将复合在一起的针头和数据进行分离。该模块的工作原理是：首先，将三个输入端口从上到下编号为0,1,2， 输出端编号类似。0 号端口连续输入去除载波频偏的数据流，当 1 号端口（定时信息）输入 1 时，也就是功能被触发，则输出端口 0 输出帧头，而数据(Payload)则保持不动。直到输入端口 2 接收到解码后的帧头信息，输出端口才有数据输出，输出数据为帧头和数据 payload 的合并数据，也就是说输出端口 1 中的信息包含一帧数据中的完整信息。

输入变量：

Header Length (Symbols)：3。 帧头占用的 OFDM 符号数。

Items per symbol: fft_len。 每个 OFDM 符号中所包含的项数。

Guard Interval (iterms): fft_len/4。一个符号中保护间隔所包含的项数。

Length tag key: length_tag_key。一帧数据的长度，它的值是 frame_len 设定为 960.

Timing tag key: "rx_time"。给每个定时点贴上标签，以便后面输出，或调试。

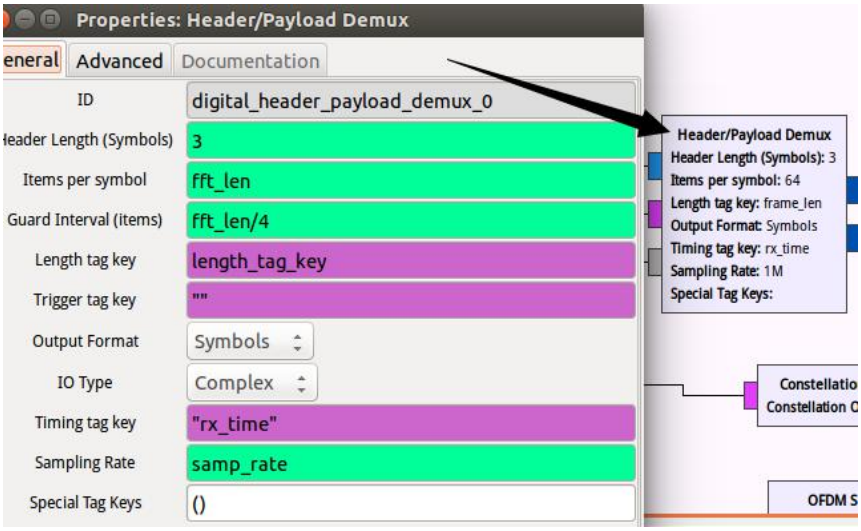
Sampling Rate: samp_rate。采样率。

可选变量：

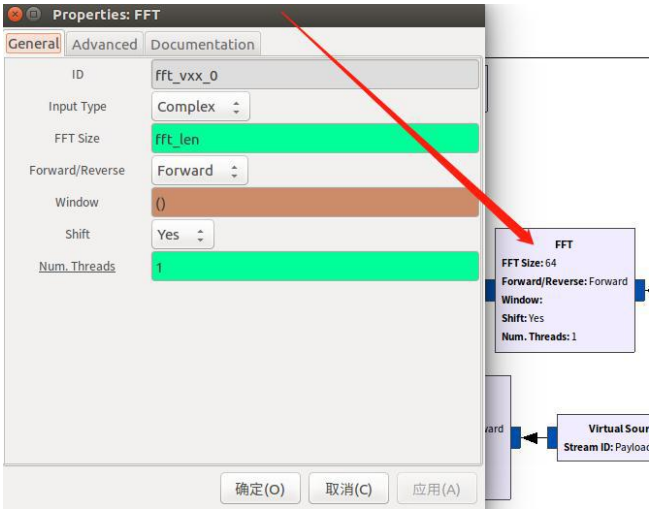
Output Format： symbols。 输出格式为符号；

IO Type: Complex.。输出数据类型为复数。

输出：去除小数倍载波频偏的数据流，数据类型为复数。



FFT： 傅里叶变换模块。在发送端也有一个 FFT 变换模块，那是逆向（Reverse）变换，为了实现解调这里应该做正向（Forward）变换。注意为与发送端一致需要选择 shift 操作，否则无法正常解调信号。



OFDM Channel Estimation：信道估计模块。该模块实现两个功能，得到整数倍频偏和系统频域响应（即信道的频率响应）。整数倍频偏的获取依靠同步字 2，其具体算法参考文献【1】。它在 FFT 模块之后，因此实现的是信道频域估计，得到的是信道的频域响应 H(k)，

$$H(k) = \frac{Y(k)}{X(k)} \quad (2)$$

其中，X(k)是同步字序列（同步字 1 或同步字 2），Y(k)是 X(k)经过傅里叶正反变换和信道之后的序列。综上，该模块的工作原理是，根据同步字 1 和 2 来获取整数倍频偏和信道的频率响应，并将这两个值作为标签传递到该模块以后的所有模块中，而输入信号流不做任何处理输出。

输入变量：

Sync. symbol 1: sync_word 1。 同步字 1。sync_word 1 是个变量，它的值在流图变量模块中给出。

Sync. symbol 2: sync_word 2。同步字 2。

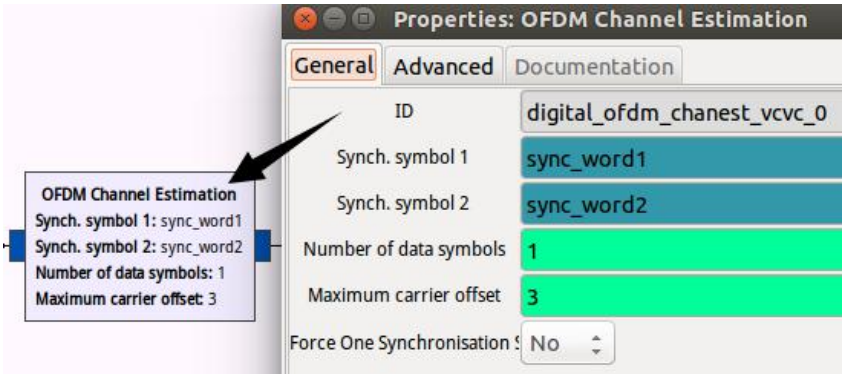
Number of data symbol: 1。 数据符号的个数，为 1.

Maximum carrier offset: 3。 最大的载波频偏，为 3，也就说允许最大有 3 个载波位置的偏移。

可选变量：

Force One Synchronizations Symbol：No。

输出： 输出数据与输入数据相比少了两个同步字，但第一数据符号多了两个标签：一个是整数倍载波频偏，记为‘ofdm_sync_carr_offset (integer)’；另一个是信号频率响应，记为‘ofdm_sync_eq_offset (complex vector)’。也就是说输出是带标签的数据流，数据类型是复数。



OFDM Frame Equalizer： OFDM 帧均衡模块。该模块实现两个功能，第一纠正整数倍频偏，第二对接收信号进行均衡。纠正整数倍频偏可以通过循环移位子载波编号来完成。比如，设接收信号经 FFT 后为 Y(k)，其产生的整数倍频偏为 n（该值在前一模块中求得，并通过标签‘ofdm_sync_carr_offset’传递到当前模块），则通过循环移位得到 $[Y(\text{fft_len}-n+1:N), Y(1:\text{fft_len}-n)]$ 来消除频偏，其中 a:b 表示 a 到 b 步进为 1 的向量，且 $a<b$。另外，信道均衡依赖系统频率响应 H(k)（该值也是从上一个模块中获得的），均衡后的信号 $Y(k)=H(k)X(k)$。

输入变量：

FFT length：fft_len。 傅里叶变换长度。

CP length: fft_len/4。 OFDM 保护间隔长度

Equalizer: header_equalizer.base()。 采用的均衡方法，内部指定。

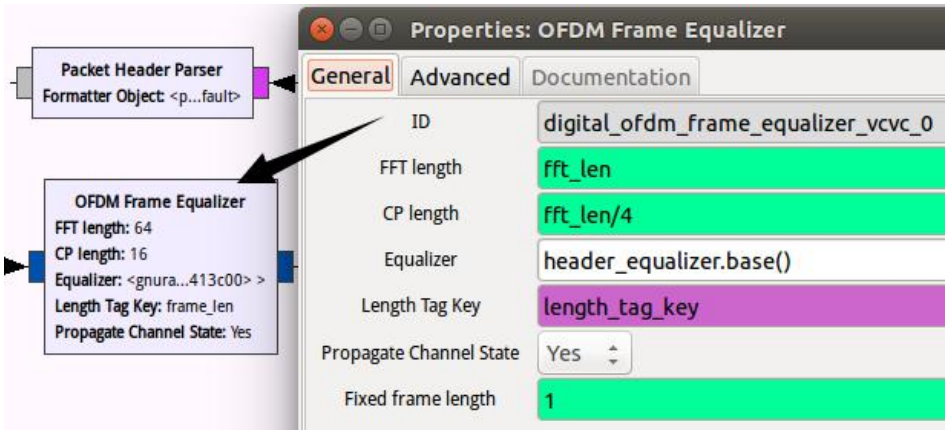
Length Tag Key: length_tag_key。 一帧数据的长度。

Fixed frame length: 1。 帧的长度是否等长，设为 1 等长。

可选变量：

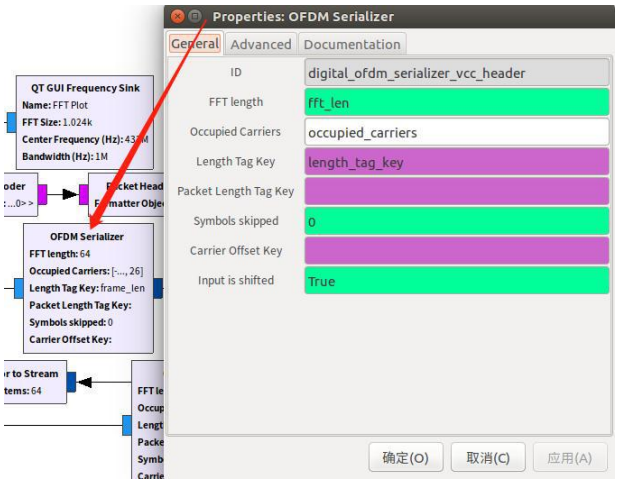
Propagate Channel State： Yes。 将信道状态作为标签继续传递下去。

输出：去除整数倍载波频偏和均衡后的带有标签的数据流，数据类型为复数。



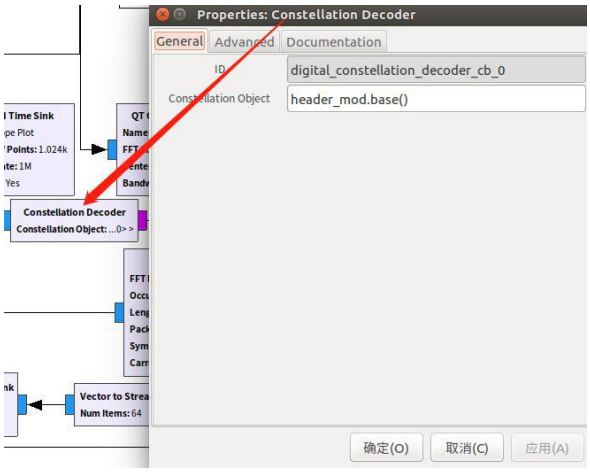
OFDM Serializer： 逆 OFDM 子载波分配模块，也称为并串转换模块。该模块实现的功能是发送端 OFDM Carrier Allocator 模块的逆过程，需要知道 OFDM 符号中每个子载波的分配规则，找到携带数据的 48 个子载波，并提取其信息。

输出： 有效数据流，去除了空载波和虚拟载波信息，数据类型复数。



Constellation Decoder： 解符号映射模块。这里解调的是针头数据，因此根据发送端规则，该模块是解 BPSK 映射。

输出： 输出的 0 和 1 的信息流，数据类型是字节。



Packet Header parser： 解帧头信息模块。该模块实现的功能是发送端 packet header generator 模块的逆过程，唯一不同的是这里不是输出一连串的 0,1 数据流，而是 PMT 字典格式的信息，伴随着一个“header_data”的 ID。

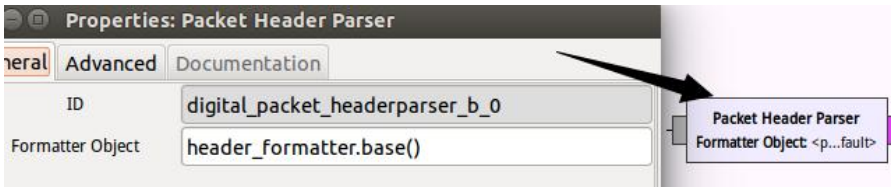
输入变量：

Formatter Object：header_formatter.base()。 帧的格式，内部指定的格式，当无输入时，调用默认格式。

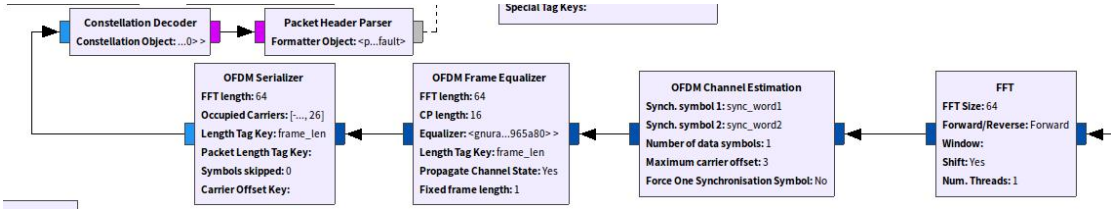
可选变量：

Propagate Channel State： 无。

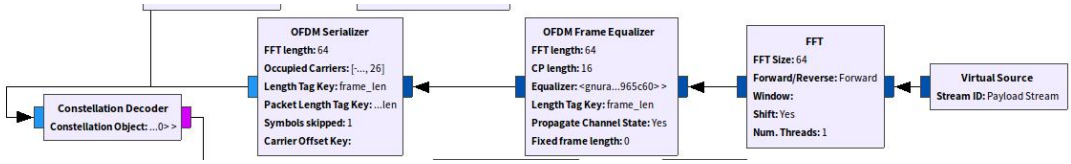
输出： PMT 字典格式的信息，伴随着一个“header_data”的 ID。



下半部分的 FFT、OFDM Frame Equalizer、OFDM seriallzer、Constellation Decoder 模块功能与之前的类似，只不过一个是解帧头的部分，一个是解数据流的部分



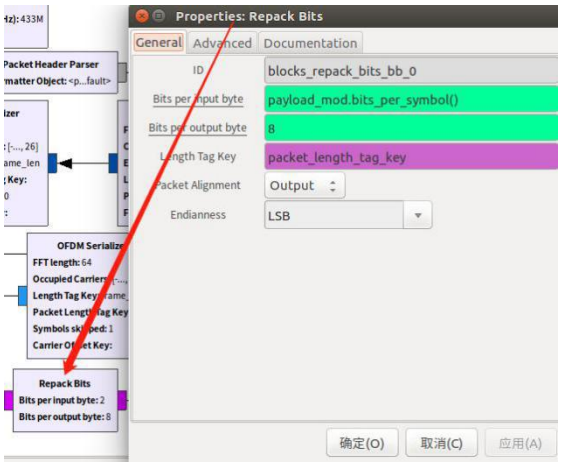
解帧头



解数据流

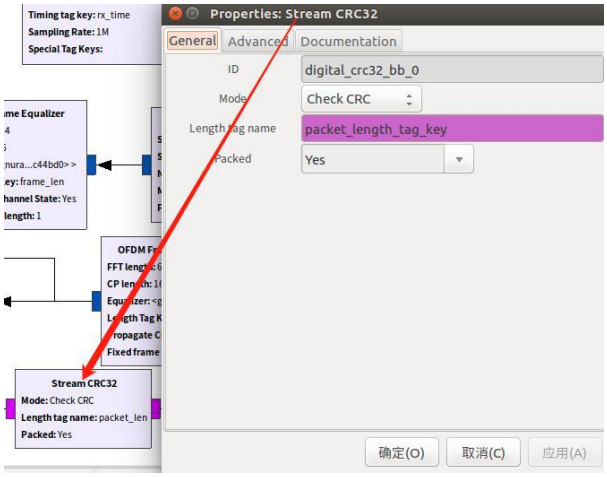
Repack Bits： 字节重组合模块。 这里实现的功能是将 2 个比特的字节变换成 8 个比特的字节，从最低位开始。

输出： 输出的 0 和 1 的信息流，数据类型是字节。



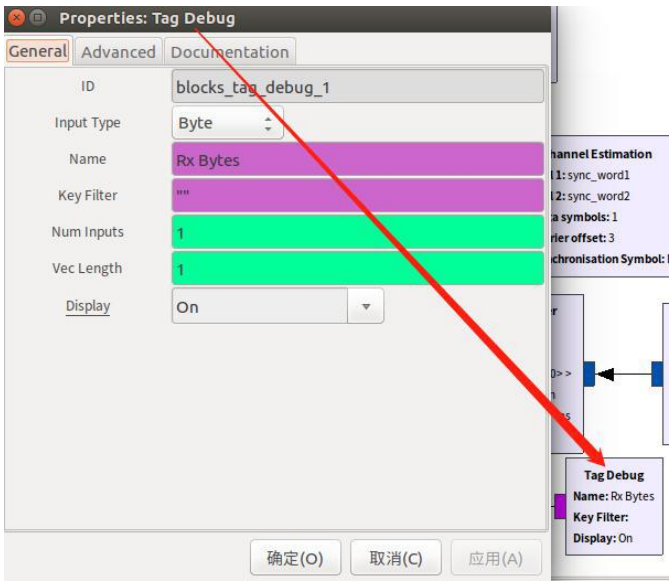
Stream CRC32： CRC 校验模块。 是发送端的 CRC 校验模块的逆过程，发送端是生成 4 个字节的校验码，而这里是验证校验码。

输出： 去除 CRC 校验字的 0 和 1 的信息流，数据类型是字节。



Tag Debug： 标签调试。没有实际的功能，只是在终端中输出数据流所携带的标签信息。

输出： 标签信息。



五、 上手学习指导（工作栈）

1. 如何安装 gnuradio？

安装方法：安装方法

在 ubuntu 16.04 上安装，经由张浩学长的测试，使用 uhd 3.15.0 版本与 gnuradio 3.7.13.5 版本运行流畅；

在 ubuntu 18.04 上安装，经过我（张腾宇）的测试，使用 uhd 3.15.0 版本与 gnuradio 3.8.2.0 版本运行流畅。

但是值得注意的是，两个版本中一些宏常量的定义是不同的！最经典的例子就是，ubuntu 16.04 上，QPSK 星座图映射出来的四个标准点距离原点的距离是 1，但是 ubuntu 18.04 上，QPSK 星座图映射出来的四个标准点距离原点的距离是 2！

2. 如何跑通 demo 并通过小项目上手 gnuradio？

这里推荐两个学习网站：

第一个，官方网站，这个网站上从入门的简单介绍、到 demo 的使用方法甚至精进一些，会告诉你很多 gnuradio 底层源代码的代码逻辑，对于大家从小白到大神有很大的帮助，常读常新；（唯一可能存在的缺点就是没有中文版？）

第二个，中文辅导资料，这个网站上从教你搭建开始，到跑通 demo 为止，比较适合英文看崩了并且仅仅是入个门而已的水平，要是希望能够动手改代码、自己写模块，建议就不用花费太多时间在这个上面了；

第三个，源代码，想要真正精通，读懂源代码是必经之路，毕竟自古华山一条路，读通源代码才能真正理解编纂 gnuradio 的通信大神们的精髓所在。

3. 如何阅读源码成为一代宗师？

静下心来，首先从配置环境开始；配好环境之后，先学习一下怎么画出正确的流程图，怎么把里面的 demo 调通；对于流程图精通之后，阅读本文档或者源代码，理解 gnuradio 的代码原理；最后上手写自己的模块，自己真正的把 gnuradio 编程得心应手的工具。

六、 如何将 SISO 系统改写成为 MIMO 系统

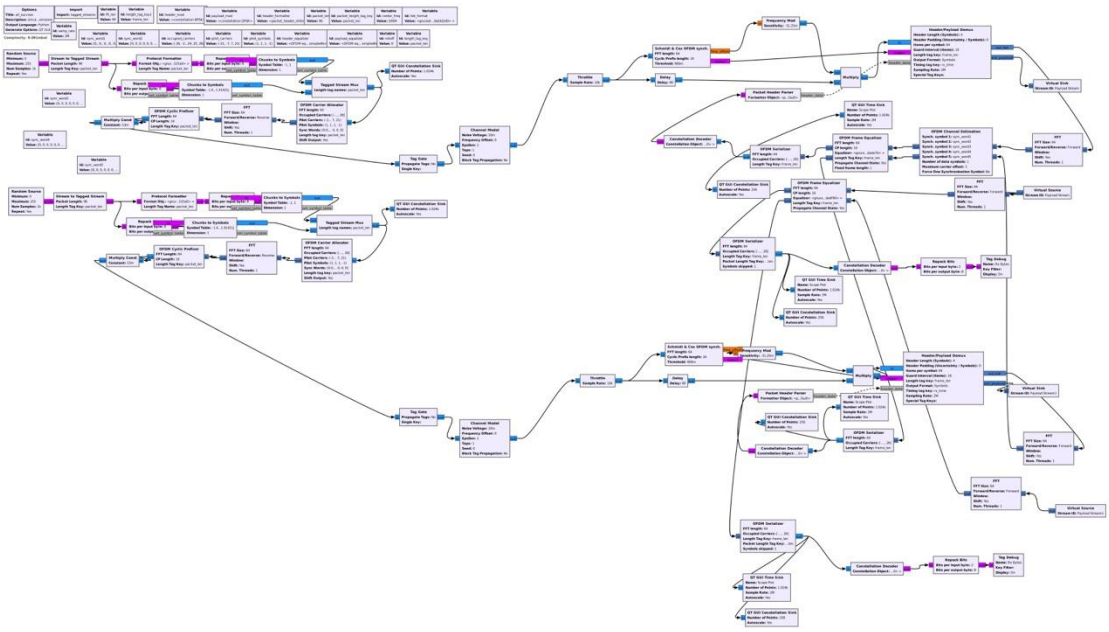


Fig 34. 经过本次修改之后的 flowgraph 整体图，在发送端没有任何修改，在接收端的修改比较明显，首先是将 equalizer 改成是双入双出、其次是将 estimation 改成双入双出。初次之外，在其余部分没有调整。这里使用的信道是两个 SISO 信道，实际上应当是使用正规的 MIMO 信道。但是由于噪声过大，使用这种方式加射频线可以产生尚可接受的效果。

1. 理论基础

本部分主要讲述 MIMO-OFDM 通信的基本原理部分，由于是本人（张腾宇）的个人理解，本人本科就读于自动化专业，通过本平台上手通信系统，更多的是就事论事，并不能将原理的最本质呈现出来。

1.1 信道模型

在通信体系中产生各种调制、信道估计和信道均衡、同步的方法，其本身上的原因是信道不够理想造成的。大家试想，如果信道完全理想：传输距离很小（忽略传输延时）、传输的过程中没有加性乘性噪声、传输无衰减、无随机过程、带宽使用从 0 到无穷的全带宽且各频段没有其他信号扰动，那么输入到输出就不需要任何方法进行处理，直接传过去就是最好的，也没有咱们啥事情了。

之所以还需要咱们，主要的原因就是信道不够理想。下面从信道的基础开始，介绍道 MIMO 的信道模型。

提到信道，咱们首先可能想到的是：电磁波在空间中怎么传播，根据这个电动力学的基本知识（好比使用菲涅尔传播模型等等物理方法）构建出一个信道真正的衰减模型。

但是实际上，上面说的方法，在自动化科学里面叫做“**机理建模**”，机理建模需要对于基本原理有很深刻的了解，就是通过物理模型硬算出来。实际上如果真的能硬算出来一个方程，说我这个方程就是无线通信传播的方程，那实际上也根本不需要咱们加什么信道估计了！为啥呢？咱有了方程，反解还能做不出来？

实际上，咱们求不出来机理建模。这些信道的实际情况是非常复杂的，就算是求出来了未必有反解的公式。

那怎么办？自动化科学里面有另一种方法，叫做“**系统辨识**”。

系统辨识就是说，我假设系统能够被某一个数学模型表示。但是我不去求去这个模型中函数的具体形式，而是在一定的时刻对系统的输入和输出进行测量，根据测量值，求出模型中的参数。

使用这种方法构建信道模型时，带来的好处就是：我根本不需要知道电磁波的传播方程，也同样能够进行效果极佳的通信。（当然，随着科技发展，机理建模对于系统的优化的重要性也逐渐显现出来了。）

所以说，咱们做系统信道估计，实际上还是在做自动化的“系统辨识”。好了，现在打通了这一重关系，我们下面讲述信道的基本辨识模型。

1.1.1. 调制信道模型

依据上面说的系统辨识方法，产生了两种基本的模型分类——调制信道模型和编码信道模型。调制信道模型是**从调制器输出、到解调器输入之间的信道**；编码信道是**从编码器输出、到解码器输入之间的信道**。

在本项目中，我们现在编码和解码都是确定的，我们选择的是调制信道模型去描述真正的物理信道。

首先咱们从调制信道开始。

从这里开始的所有内容是针对一个子载波而言的！！不涉及多个子载波！！也就是说，下面的内容是同一个频率的信号！！

本质上而言，可以把信道视为是一个函数：

$$e_o(t) = f[e_i(t)] + n(t) \quad (1)$$

这个信道函数，为了方便数学分析和计算，常常将这个信道函数 f 构建为如下形式：

$$e_o(t) = k(t)e_i(t) + n(t) \quad (2)$$

这个基础的模型就是调制模型，k 函数具有很复杂的数学形式，在精致的构造后，这个模型可以表示出信道的线性失真、非线性失真、时间延迟以及衰减等等效果。

至于如何构造这个函数，就涉及机理建模的东西了，我们这里避免这么做。

为了避免使用物理计算求出 k，我们现在引入**随参**和**恒参**的概念。随参信道是指信道的特性随着时间是一个**随机过程**，也即是不断变换的；恒参信道是指信道的特性随时间是不变的。

我们虽然无法保证咱们的信道是稳定的恒参信道，但是在实验室环境中，大部分情形下还是能保持一个 symbol 传输的时间间隔内是恒参信道的。

所以这里的 k(t)实际上对于某一个时刻就是一个数字！（当然这里是说 SISO，如果是 MIMO 的话 k 就是一个数字矩阵）

因为是一个数字，所以就直接反求出 k 就是信道估计，进而可以做信道均衡。非常爽！

1.1.2. MIMO 信道模型

从这里开始的所有内容是针对一个子载波而言的！！不涉及多个子载波！！也就是说，下面的内容是同一个频率的信号！！（而且在(3)式中，所有天线仅仅发送了一个 symbol，也因此E_o矩阵高是 n，宽是 1 了）

前面铺垫了那么多，现在接着使用咱们的“系统辨识”模型，把它拓展到 MIMO **(m transceiver antenna with n receiver antenna)**里面应该长啥样呢？应该是如下的矩阵形式：（推导过程如下图，公式如下定义）

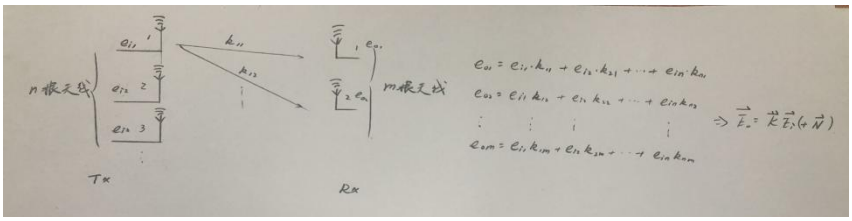


Fig 35. MIMO 信道的系统辨识模型的传输过程的数学推导。

$$\boldsymbol{E_o}(t) = \boldsymbol{K}(t)\boldsymbol{E_i}(t) + \boldsymbol{N}(t) \quad (3)$$

$\boldsymbol{E_o} \in \boldsymbol{Complex}^{n\times 1}$；$\boldsymbol{E_i} \in \boldsymbol{Complex}^{m\times 1}$；$\boldsymbol{K} \in \boldsymbol{Complex}^{n\times m}$；

$$N \in Complex^{n\times 1}$$

那这个对于某一个时刻是什么样子呢？

$$E_o = KE_i + N \quad (4)$$

在咱们做的时候，咱们认为信道里面没有 N，所以上面的(4)简化为：

$$E_o = KE_i \quad (5)$$

好啦，我们现在有了 MIMO 信道的“系统辨识”建模了，非常爽，兄弟们咱们离成功又近了！

信道的模型就讲到这里，最后再提醒一遍：以上**的所有内容是针对一个子载波而言的！！不涉及多个子载波！！也就是说，上面的内容是同一个频率的信号！！**

1.2 同步方式

说一说同步。首先我们搞明白一个问题：同步是干啥的呢？下面几段我先回答同步的作用，之后每一个小节分别讲解 SISO、MIMO 的常见同步方法讲解。

好，我们从这里回答同步的意义、分类。

我们的发送方和接收方是需要进行同步的，因为首先，**发送方和接收方所使用的时钟频率是不同的，发送方连续发 2 个“1”，接收方按照自己的时钟来看可能是 10 个“1”（或 1 个“1”），这样绝对不行**；**或者，即便不会出现上述的情形，也有可能因为信道的延迟（多径效应等等）的问题造成接收的时候产生帧的开头没法对齐的问题**；**还有可能因为信道干扰产生频率偏移，子载波之间相互干扰等等**。

从上述的几个原因考虑，我们需要同步技术。而且上面这一段所举的例子都是有所用意的：第一、二种情形是时间方面的“同步”；第三种情形则是频率方面的“同步”。（第一种黄色，第二种绿色，第三种青色）

到这里我们就可以讲述同步技术的定义了：OFDM 同步技术主要包含**频率同步**和**时间同步**。其中，时间同步包含**采样时钟同步**和**符号定时同步**。

定义里面说的“采样时钟同步”是解决“第一种情形”的；“符号定时同步”是解决“第二种情形”；“频率同步”是解决“第三种情形”。

下面讲一下同步技术的分类。

从模型的不同这个角度去区分的话，同步技术主要分成两类，第一类是“**集中式 MIMO-OFDM 同步模型**”;第二类是“**分布式 MIMO-OFDM 同步模型**”。

第一种“集中式”模型的关键在于基站 BS 和移动端 MS 的天线都是集中在一起的，因此这个模型中只有一个时延、一个频偏。自 2000 年以来，针对这种模型的研究是比较丰富的，好比 Mody&Stuber 的同步方法、基于 Zadoff-Chu 序列的同步方法等等。

第二种“分布式”模型的关键在于发送端基站 BS 采用分布式天线，移动端 MS 采用集中式天线。因此这种模型中，接收端天线对于不同的发送端天线具有不同时延、频偏。2000 年以来，针对这种模型的研究较少，我们这里也就不去钻这个牛角尖。【注：这种模型的难点在于：如何设计一种方式来区分不同发送方天线上的信号】

从同步方法的不同这个角度区分的话，同步技术主要分为两类，第一类是“**数据辅助的同步方法**”；第二种是“**盲同步方法**”。

第一种“**数据辅助的方法**”采用的方式是**巧妙构建“训练序列”**。构建的“训练序列”主要需要满足**两个要求**：其一，**训练序列正交**（OFDM 就是正交的）;

由于“数据辅助的方法”十分重要，我在这里讲一下这种方法一般包含的步骤：

----------手动分割线----------

首先，**时间粗同步**。这个步骤通过接收信号做延迟自相关后取模来实现。这样说可能大家不理解，举用 Schmidl&Cox 方法中的例子就是：使用构造出的训练序列的前 T/2 与后 T/2 做自相关运算，如果自相关的结果大于某个阈值，那么判断这里就是接收信号的起始位置。（如果说到这里还是不理解，建议先阅读一下 1.3.1 里面的“SISO 系统同步”部分）

第二，**频率粗同步**。将接收训练序列做 FFT 之后与本地训练序列做相关，求最大值。这里还是举用 Schmidl&Cox 的例子，就是对应于 Schmidl&Cox 方法中消除“整数频偏”的部分。

第三，**频率精同步**。将接收训练序列做延迟自相关后取自相关结果的相位。这里对应于 Schmidl&Cox 方法的消除“小数频偏”的部分。

第四，**时间精同步**。通过接受训练序列与本地序列相关得到。Schmidl&Cox 里面没有这一步，但是在 Zadoff-Chu 里面有的。

----------手动分割线----------

其二,**满足强位移自相关性**(好比 schmidl&cox 方法里面构建的 T/2 周期序列)。【当然，应该指出，不同的方法构建出来的序列还有其他各自不同的特性，但是上面说的这两个是**务必满足**的】

第二种“**盲同步方法**”不构造训练序列，而是直接**巧妙构造“保护间隔”**。这种方法估算效果不好，很少使用，即便是简化了编码、节约了带宽资源。

以上就是对于同步方法的简单介绍。

下面将会结合本次的项目，分为“SISO 同步”和“MIMO 同步”两个模块去讲，在“SISO 同步”中会介绍 Schmidl&Cox 方法；在“MIMO 同步”中会介绍 Zadoff-Chu 序列方法。

本项目 SISO 里面使用的是 Schmidl&Cox 设计的方法,这个方法非常感谢胡冠杰学长、杜英杰学长以及刘晰鸣同学的文献阅读，基本理论梳理非常详细。

本项目 MIMO 里面使用的仍然是照搬 Schmidl&Cox 在接收方的两路分别做，虽然在理论上不尽正确，但是考虑到咱们实验环境是天线间距极近、信道间没有干扰，所以照搬 Schmidl&Cox 并没有出现非常大的问题。当然，下面在介绍同步方法的时候还是介绍了正规的 MIMO 同步方法——Zadoff-Chu 序列。请之后的开发者使用 Zadoff-Chu 序列法进行同步为盼。

1.2.1. SISO 系统的同步

SISO 系统的同步这里直接介绍 schmidl&cox 方法。由于学长们和晰鸣的数学推导十分详细，我这里简单讲一下直观上的理解。

Schmidl&cox 讲的是什么故事呢？是这样，我从发送端发出来一个调制好的 symbol，接收端显然是能够收到的。但是世界上的信号错综复杂，接收方并不知道哪里才是发送方信号的起始,由于并不知道发送方的起始位置，自然也就不知道怎么算接收到信号了。

所以怎么办呢？schmidl&cox 说：那我构造一个特别特殊的信号，使得接收端知道啥时候开始不就行了？

Ok，说干就干，构造的信号是以 T/2 为周期的子载波，这些子载波构成一个 symbol 就是想要的“特殊信号”。为啥嘞？因为这种信号在前 T/2 部分与后 T/2 部分去做互相关运算的时候，得到的结果是非常大的！（一般来讲，数理统计学中讲到，没啥关系的两个信号，做互相关都是 0，但是完全一致的信号做互相关不是 0，而是一个比较大的常数）。那就非常爽！

接收端可以始终对于接收的信号做前 T/2 和后 T/2 的互相关运算，设定一个阈值，当互相关运算的结果大于这个阈值的时候，就认为是接收到信号了。

Perfect！我们现在解决了信号在哪里开始的问题。下面是“同步”需要解决的第二个问题：如何估计频偏？

Schmidl&cox 说：有了我们刚刚提到的设计出来的 T/2 周期的子载波构成的 symbol，非常好实现。

频偏一般包含两个部分，小数频偏和整数频偏。小数频偏在我们现在的环境下是主要的，一般在好几公里的传输过程中才会有整数频偏。

我们先说说小数频偏的故事。求法非常简单，就是刚刚上面说的，求出来前 T/2 与后 T/2 的互相关的值，这个值的角度就是小数频偏。推导请参考原论文，直观理解那就非常好理解，靠直觉也会觉得是这么算【手动捂脸.....】

下面讲一下整数频偏的故事。作者给出了一个判定准则如下式：

$$B(g) = \frac{\left|\sum_{k\in X} x_{1,k+2g}^{*} v_k^{*} x_{2,k+2g}\right|^2}{2\left(\sum_{k\in X}|x_{2,k}|^2\right)^2}$$

让这个 B 最大的 g 就是整数频偏。具体的参考原论文。

上述的就是我简单描述了 SISO 系统里面的 schmidl&cox 的同步方法，挺好玩的，而且实际上也挺好用的。咱们 gnuradio 用的就是这个系统。还不错。

1.2.2. MIMO 系统的同步

上到 MIMO 里面，大家想象一下，其实直接照搬 schmidl&cox 还是不一定对的，为啥嘞？因为直接照搬 schmidl&cox 的话，万一多天线的频偏不一样，但是接收的时候不管不同接收信号的偏移不同，直接接收，那这频偏是没法消掉的。

这里介绍标准的基于 Zadoff-Chu 训练序列的同步方法。

这里首先声明一下：下面所说的“序列”是什么意思呢？序列的意思是：这是一个模拟信号，模拟信号采样的点数是 N 个，所以这叫做训练序列。其实就是一个子载波对应的 Symbol，这个 Symbol 被采样了 N 个点。

(1) **训练符号设计方法** [1]

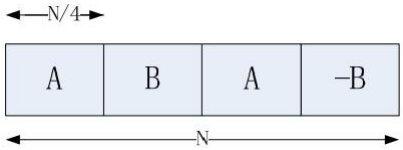


Fig 36. Zadoff-Chu 序列设计。

将总长为 N 的训练序列分为 N/4 等长的四段，其间关系如上图 Fig 36.所示，每一个 N/4 段中的构造方式如下式：

$$Z_r(m) = \begin{cases} e^{-j\frac{2\pi r}{M}\left(\frac{m^2}{2}+qm\right)}, \\ e^{-j\frac{2\pi r}{M}\left(\frac{m(m+1)}{2}+qm\right)} \end{cases}$$

上式中，M 为序列长度，这里是 N/4；m=0,1,2,...,M-1；q 为任意整数；r 为 Symbol 序号，且与 M 互质。

对于每一根天线，这个序列是不同的，对于不同的天线 A 与天线 B，进行间隔为(p-1)*9 个采样点的循环移位！（p 表示间隔了 p 根发射天线）

【注：在 matlab 中存在一个函数 seq=zadoffChuSeq(R, N)，这个函数自动构造 Symbol 序号为 R，长度为 N 的 Zadoff-Chu 序列】

(2) **同步方法第一步：时间粗同步**

$$\Phi_{q,n}=\frac{\left|\varphi_{q,n}\right|^2}{(P_{q,n})^2}$$

其中:

$$\varphi_{q,n}=\sum_{k=0}^{N/4-1}\left(r_{q,n+k}^{*}r_{q,n+k+N/2}-r_{q,n+k+N/4}^{*}r_{q,n+k+3N/4}\right)$$

$$P_{q,n}=\sum_{k=0}^{N/4-1}\left(\left|r_{q,n+k}\right|^2+\left|r_{q,n+k+N/4}\right|^2+\left|r_{q,n+k+N/2}\right|^2+\left|r_{q,n+k+3N/4}\right|^2\right)$$

上述过程中计算出的$\Phi_{q,n}$取为最大值的时候就是接收信号开始的时刻。这里计算出的开始时刻其实就是下面使用的d_{opt}。

(3) **同步方法第二步：频率精同步（估计小数频偏）**

$$\hat{\varepsilon}_{q,f,1}=\frac{1}{\pi}\tan^{-1}\left[\frac{\sum_{k=1}^{N/4}\mathrm{Im}\left[r_{q,d_{opt}+k}^{*}\cdot r_{q,d_{opt}+k+N/2}\right]}{\sum_{k=1}^{N/4}\mathrm{Re}\left[r_{q,d_{opt}+k}^{*}\cdot r_{q,d_{opt}+k+N/2}\right]}\right] \quad (3.42)$$

$$\hat{\varepsilon}_{q,f,2}=\frac{1}{\pi}\tan^{-1}\left[\frac{\sum_{k=1}^{N/4}\mathrm{Im}\left[r_{q,d_{opt}+k+N/4}^{*}\cdot(-r_{q,d_{opt}+k+3N/4})\right]}{\sum_{k=1}^{N/4}\mathrm{Re}\left[r_{q,d_{opt}+k+N/4}^{*}\cdot(-r_{q,d_{opt}+k+3N/4})\right]}\right] \quad (3.43)$$

$$\hat{\varepsilon}_{q,f}=\frac{1}{2}(\hat{\varepsilon}_{q,f,1}+\hat{\varepsilon}_{q,f,2}) \quad (3.44)$$

这里$\hat{\varepsilon}_{q,f,1}$，$\hat{\varepsilon}_{q,f,2}$为接收天线$q$根据自身重复的 A 和重复的 B 估计获得的频偏，$\hat{\varepsilon}_{q,f}$是它们的平均，最后对接收天线进行等增益合并：

$$\hat{\varepsilon}_{f}=\frac{1}{N_r}\sum_{q=1}^{N_r}\hat{\varepsilon}_{q,f} \quad (3.45)$$

这里，d_{opt}表示粗帧同步点，因为相位取值在$[-\pi,\pi]$范围内，所以$\hat{\varepsilon}_{f}$取值在$[-1,1]$区间。

(4) **同步方法第三步：频率粗同步（估计整数频偏）**

由于整数倍频偏会引起 FFT 之后的频域数据发生循环移位，因此，整数频偏估计在频域进行，将时域天线 p 发送的训练序列 $\{s_p\}_{n=0}^{N-1}$ 做 N 点 FFT 后所得的频域序列 $\{S_p\}_{n=0}^{N-1}$ 与天线 q 接收的频域解调信号 $R_{q,n}$ 做循环互相关，可得整数倍频偏：

$$\chi_{q,k}=\sum_{n=0}^{N-1}S_{p,(k+n)N}^{*}R_{q,n} \tag{3.46}$$

$$\hat{\varepsilon}_{q,\mathrm{int}}=\arg\max_{k}\left\{\left|\chi_{q,k}\right|\right\},k=0,1,\cdots,N-1 \tag{3.47}$$

需要注意的是，帧检测的准确性以及小数频偏估计的精度都会影响整数频偏估计的可靠性。因为若残留的小数倍频偏较大，势必会引起严重的载波间干扰，使得频域整数倍频偏估计不准确；同时若帧检测的位置不正确，使得接收的频域解调信号 $R_{q,n}$ 不包含完整的频域训练序列，那将会出现错误的整数倍频偏估计结果。最后，在完成频偏估计后，对数据进行频偏补偿。

(5) **同步方法第四步：时间精同步**

完成接收信号的粗定时同步、小数倍频偏估计和整数倍频偏估计之后，最后进行符号定时细同步，FFT 窗的位置由细同步确定。细同步由时域的本地序列和接收到的信号进行相关可得：

$$\Psi_{p,q,n}=\sum_{p=1}^{N_t}\frac{\left|\psi_{p,q,n}\right|^2}{(P'_{q,n})^2} \tag{3.48}$$

$$\psi_{p,q,n}=\sum_{k=0}^{N-1}s_{p,k}^{*}r_{q,n+k} \tag{3.49}$$

$$P'_{q,n}=\sum_{k=0}^{N-1}\left|r_{q,n+k}\right|^2 \tag{3.50}$$

其中，p 和 q 分别表示发送与接收天线，$s_{p,k}$ 表示第 p 个发射天线发送的第 k 个样点，构成本地序列，$r_{q,n}$ 为接收天线 q 接收到的第 n 个样点。$\Psi_{p,q,n}$ 取最大值时，对应的时刻 n 即为细同步定时点：

$$n_{p,q,fine}=\arg\max_{n}\left\{\Psi_{p,q,n}\right\} \tag{3.51}$$

1.3 信道估计方式

这一部分在项目中的分析相对于上面的同步部分确实简单了一些。（当然实际上这一部分是很复杂的，可以上机器学习方法，这里只是说我们使用的方法很简单，是最小二乘方法）

下面对咱们使用的方法进行介绍。

由

$$\boldsymbol{E_o=KE_i} \quad (5)$$

我们可以发现，实际上如果希望求 K，那么直接如下式所示就行了：

$$K = E_o E_i^+$$

上式中的 E_i^+ 就是 E_i 的广义逆。结束。

1.4 信道均衡方式

这一部分在本次项目中也是非常简单的。这一部分直接就是拿我们预测出的 K 与接收矩阵相乘就行了。

2. 代码层软件仿真改写

这个部分我主要讲述一下代码的基本思路，关于具体的代码编写，请参考附件中的代码。

首先是修改 chanest 部分，这一部分的主要改动是将两个同步字改成了三个。原来的 SISO 中使用两个同步字，其中同步字 1 进行 Schmidl&Cox 信道同步，同步字 2 一方面用于辅助 Schmidl&Cox 进行同步，另一方面是用于进行信道估计。这里改成了三个同步字，第一个同步字仍然是用于 Schmidl&Cox 信道同步，同步字 2 一方面用于辅助 Schmidl&Cox 进行同步，另一方面，同步字 2 与同步字 3 一同进行信道估计。

至于为什么我这里使用了两个同步字（同步字 2 与同步字 3）进行信道估计，首先，由上面的(5)式子，首先还是明确：每一个子载波都是有一个独特的信道矩阵的。如果这个子载波是 2*2 的收发 MIMO 的话，那么这个子载波的信道矩阵就应该是 2*2 矩阵。由于信道估计的本质是通过训练序列的接受、发送解方程解出信道矩阵。那初中的知识告诉我们，2*2 矩阵有 4 个变量，4 个变量需要 4 个方程才能解出来，所以每一个天线设计 2 个同步字用于信道估计，一共两个天线那接收的时候就是 4 个同步字，4 个同步字对应 4 个方程，所以恰好够解出信道矩阵。

接下来是修改 estimator 部分，这一部分实际上就是很简单，进行了 2*2 信道的信道均衡。修改的就是变成矩阵计算、除此之外，就是整数频偏矫正。整数频偏矫正这里就是直接使用了两个 Schmidl&Cox 估计出来的整数频偏的平均值。还是那句话，这个方法虽然不严谨，但是问题其实也不太大。

剩下的在代码里面的注释写得很清楚了，请自行阅读。

3. 硬件层调试（本部分由张浩学长撰写）

本 demo 基于以下硬件和指定版本的系统和软件：

电脑两台；USRP X310 两台；Ubuntu18.04；UHD3.7.13.5;GNU Radio3.8.2.0。

假设你已经掌握了连接以上硬件、安装以上软件的方法，则接下来的操作步骤如下：

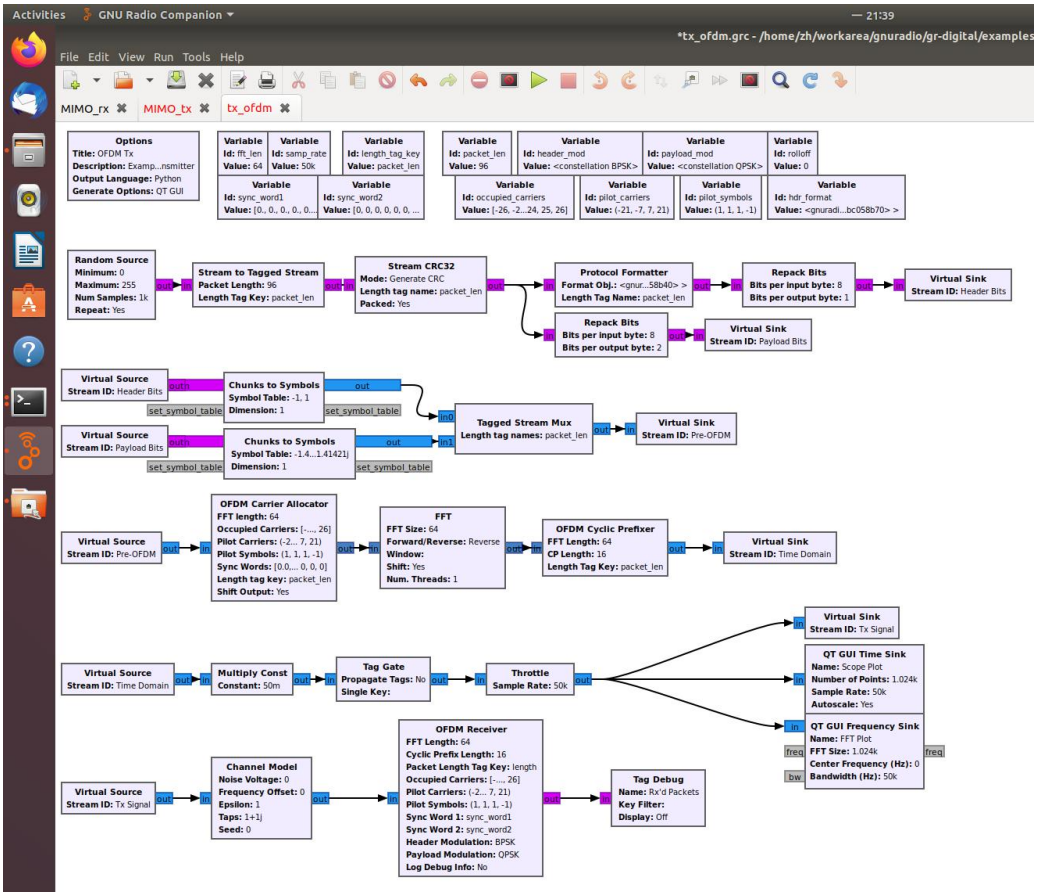
在一台电脑桌面点击右键打开终端，输入以下指令：

>>gnuradio-companion

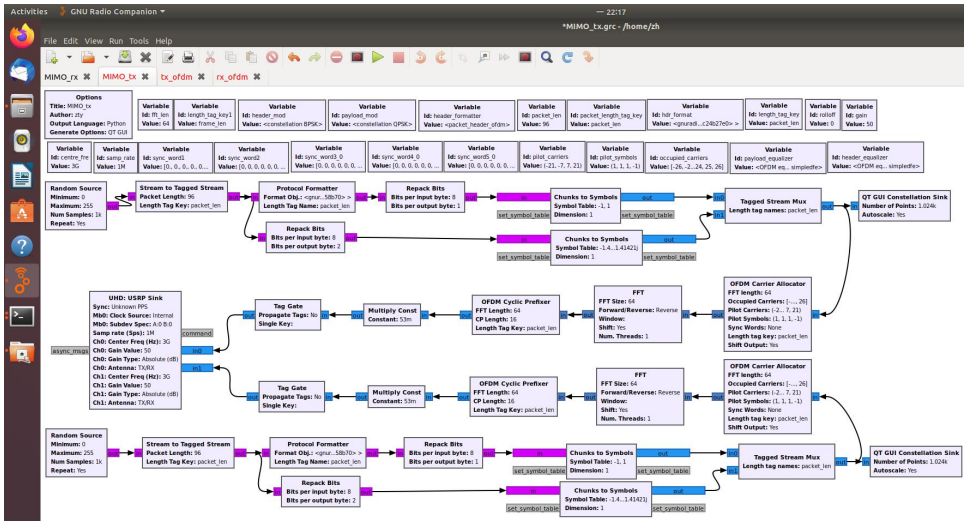
进入图形界面，打开以下位置的文件：

/gnuradio/gr-digital/example/ofdm/tx_ofdm.grc;

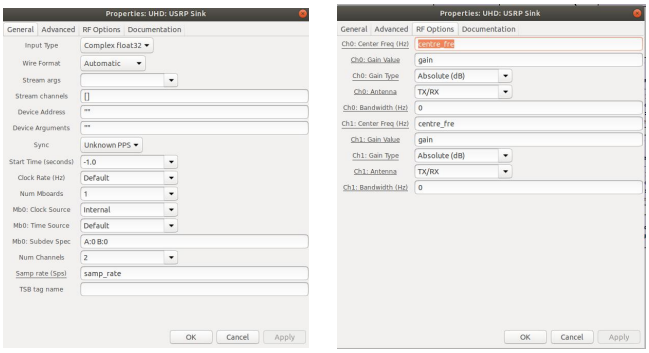
如下图所示：



我们将其改动成下图中的样子：



将其命名为 MIMO_tx，首先是变量名，就不一一介绍了，按照图中更改即可值得注意的是 sync_word3_0、sync_word4_0 和 sync_word2 取值一样，sync_word5_0 与他们相反（即每个数加上一个负号）。然后是流程，由之前的一路变为了两路，下面主要介绍下 UHD:USRP:Sink 模块的配置，如下图所示：



按照图中配置即可，其中，Mb0:Subdev Spec 取值“A:0 B:0”意思是同时打开该 USRP 的 RF0 和 RF1 射频子板。

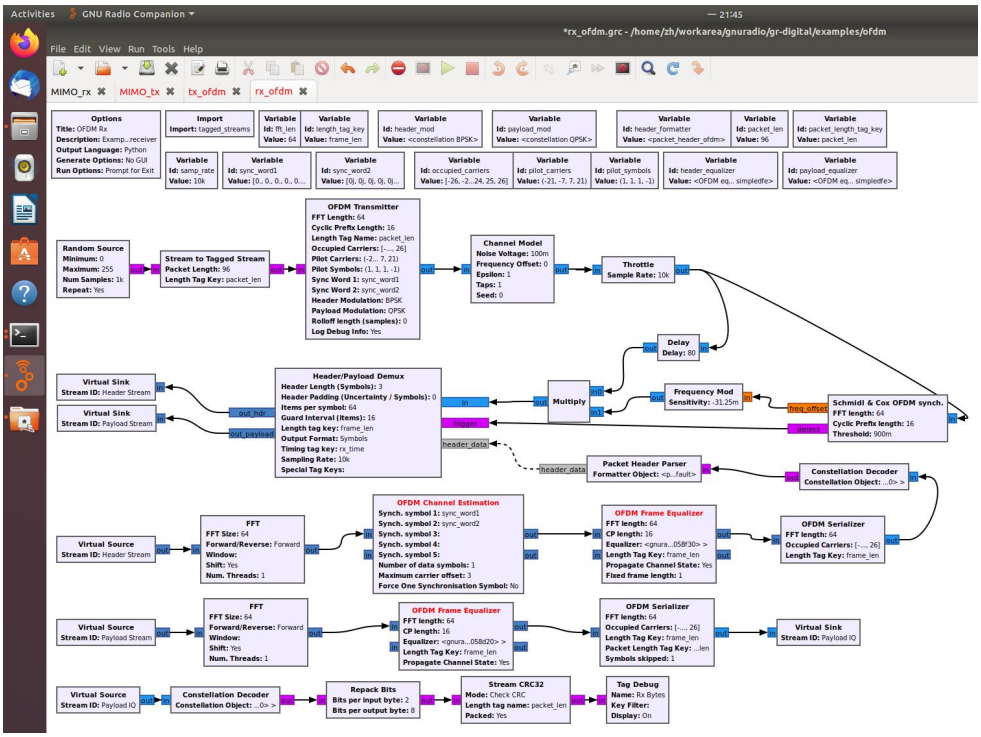
在另一台电脑桌面点击右键打开终端，输入以下指令：

gnuradio-companion

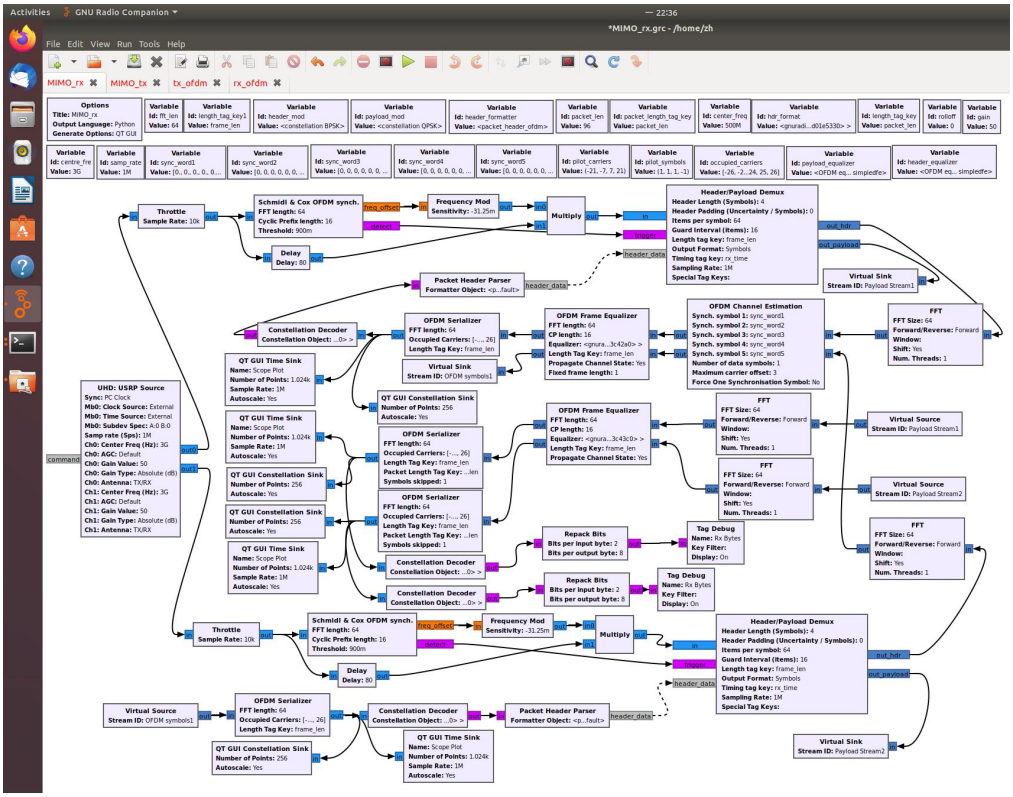
进入图形界面，打开以下位置的文件：

/gnuradio/gr-digital/example/ofdm/rx_ofdm.grc;

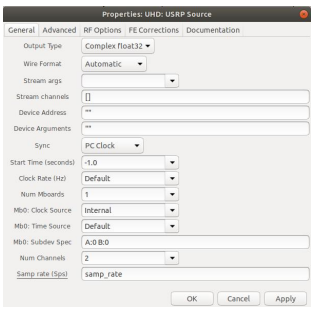
如下图所示：

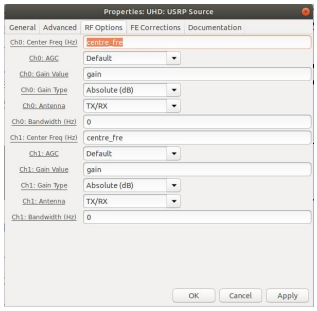


注意此时的 grc 有报错无法运行，因为我们已经更改了底层代码，有好几个模块的接口尚未连接，等我们将其接上就好了。我们将其改动成下图中的样子：



将其命名为 MIMO_rx，可以发现接收机较为复杂。变量名不再介绍，下面主要展示下 UHD:USRP Source 的配置，如下图：





可以发现其和 UHD:USRP SINK 一样。

运行后的结果基本如下图所示：

